INGEGNERIA DELL'INFORMAZIONE · SEDE DI LATINA · A.A. 2026/27

Tecniche della Programmazione

Fascicolo per preparare l'esame del corso del prof. Marco Temperini: teoria spiegata in modo semplice, codice C collaudato, le tracce d'esame risolte e quelle per esercitarsi.

22 gen 2027	2 h 30	13 + 10 + 7	ANSI C
PRIMO APPELLO	PROVA SCRITTA	PUNTI PER ESERCIZIO	LINGUAGGIO

Materiale di studio non ufficiale, costruito sul programma e sui materiali pubblicati sul sito del corso. Per regole e date fanno fede il sito e Classroom.

INTRODUZIONE

Prima di cominciare: com'è fatto l'esame

Questo fascicolo segue il **programma 2026/27** di *Tecniche della Programmazione* del prof. Marco Temperini (Ingegneria dell'Informazione, sede di Latina). L'ho costruito leggendo il sito del corso: programma, calendario delle lezioni, slide, complementi didattici, esercitazioni guidate (EG) e i testi d'esame d'esempio. Tutti i programmi C che trovi qui sono stati compilati e provati in ANSI C (`gcc -std=c89 -pedantic -Wall`), lo standard richiesto dal professore.

Le regole, in breve

COSA SERVE PER SOSTENERE L'ESAME

- **Homework**: chi si è immatricolato dal 2025/26 in poi (quindi anche tu) deve aver **consegnato e visto accettati i compiti (homework)** su Classroom. Senza homework accettati non puoi presentarti.
- **Iscrizione** alla prova scritta su **Infostud**.
- **Documento di riconoscimento** il giorno dello scritto, altrimenti non sei ammesso.
- Usa **solo l'email istituzionale** (`...@studenti.uniroma1.it`) per Classroom.

L'esame finale ha due parti, di norma in giorni diversi:

1. **Prova scritta**, di **2 ore e 30 minuti** nei testi d'esempio (la pagina degli esami dice "tipicamente alcuni esercizi in due ore"). Si scrive su carta, senza computer.
2. **Prova orale**: si discute lo scritto e possono arrivare domande su tutto il programma. La data dell'orale viene comunicata alla fine dello scritto.

Gli homework

Un homework è un programma C da scrivere su un "macro-argomento" appena studiato: tabelle con array, tabelle con liste, metodologia di sviluppo, modularizzazione, ricorsione. Il testo esce su Classroom ("Lavori del corso"). Regole del professore:

- consegnali **nell'ordine** in cui sono pubblicati (primo, secondo, ...);
- tra una consegna e l'altra devono passare **almeno 5 giorni**;
- il consiglio è consegnare ogni homework **entro una o due settimane** dalla pubblicazione. Consegnarli tutti alla fine è inutile per imparare, e rischi di non fare in tempo per l'appello (per via dei 5 giorni).

Com'è fatta la prova scritta

Ho analizzato tutti i testi d'esempio pubblicati. La struttura è **sempre la stessa**, ed è la cosa più importante di questo fascicolo:

Esercizio	Punti	Cosa chiede (sempre)
1	da 0 a **13**	Definire i **tipi** per una **lista collegata** (record + puntatori) di elementi descritti nel testo, poi scrivere una funzione `eser1` che **scorre la lista** e restituisce **più risultati** (un valore con `return`, gli altri con **parametri di output**).
2	**10**	Una funzione `eser2` che **modifica la lista**: eliminare un elemento (es. il più importante) oppure **inserire mantenendo l'ordine**.
3	da **-1** a **7**	Una domanda "di teoria applicata": **test a scatola nera / trasparente** applicato a `eser1`, oppure **rappresentazione floating point** di un numero, oppure una **funzione ricorsiva** con esecuzione simulata e **record di attivazione**, oppure (vecchio programma) **visite di un albero**.

Totale: 30 punti. Attenzione al **-1** dell'esercizio 3: una risposta sbagliata o confusa può toglierti un punto. Rispondi con ordine.

COME LEGGERE I TESTI D'ESAME

In tutti gli elementi da memorizzare c'è **una stringa "di lunghezza non nota a priori"** (descrizione, cognome e nome...). È un segnale preciso: va rappresentata come `char *` e allocata **esattamente** con `malloc(strlen(...)+1)` (capitolo 9). Ci sono poi **stringhe di lunghezza fissa** ("codice di 9 caratteri", "grado di 8 caratteri"): quelle diventano `char campo[9+1]`. Gli altri campi sono `int`, `char` o `double`.

Testi di riferimento

- **testo1**: A. Bellini, A. Guidi, *Linguaggio C*, McGraw-Hill. È il testo principale, nuovo da quest'anno: il professore preferisce che lo adottiate.
- **testo2**: D. Calvanese, P. Liberatore, F. Massacci, R. Rosati, *Programmazione di strutture dati in C*, Progetto Leonardo. Serve per **test**, **liste** (cap. 6) e **alberi** (cap. 8). Dalla lezione sulle liste in poi il corso lo usa "ufficialmente".
- In alternativa al testo1 si può usare il Deitel & Deitel (testo degli anni scorsi), seguendo le annotazioni del programma 2024/25.

Nel fascicolo, accanto a ogni capitolo, trovi tra parentesi i capitoli del testo indicati dal programma.

Le date (A.A. 2026/27, dal sito del corso)

Appello	Data e ora	Note
Gennaio	**22/01/2027**, ore 9:30, Aula 15	il primo utile per te
Febbraio	**17/02/2027**, ore 9:30, Aula 15	
Aprile (straordinario)	09/04/2027, ore 15:30	solo per categorie particolari (lavoratori, fuori corso, ...)
Giugno	09/06/2027, ore 9:30, Aula 15	
Luglio	14/07/2027, ore 9:30, Aula 15	
Settembre	16/09/2027, ore 9:30	aula da destinarsi
Ottobre (straordinario)	29/10/2027, ore 15:30	solo per categorie particolari

Le date possono cambiare: controlla sempre la pagina esami del corso e Classroom.

Come usare questo fascicolo

Il fascicolo segue l'ordine del corso. Ogni capitolo ha spiegazioni semplici, codice C commentato e provato, riquadri colorati ed esercizi con soluzione nascosta (clicca su "Soluzione" solo dopo averci provato davvero).

ALL'ESAME

Questi riquadri dicono che cosa serve **proprio per lo scritto o l'orale**.

ERRORE TIPICO

Questi riquadri mostrano gli sbagli che costano punti.

DA SAPERE

Approfondimenti e collegamenti tra argomenti.

Un piano di studio consigliato (circa 8 settimane prima di gennaio)

Settimana	Cosa studiare	Capitoli
1	Architettura, rappresentazione dei numeri, basi del C	1, 2, 3
2	Istruzioni di controllo, funzioni, record di attivazione	4, 5
3	Array, file, puntatori	6, 7, 8
4	Stringhe, allocazione dinamica, struct e tabelle	9, 10
5	Ricorsione, ricerca e ordinamento, test	11, 12, 13
6	**Liste** (tutte le operazioni, a memoria e capite)	14
7	Alberi (cenni), metodo per lo scritto, tracce risolte	15, 16, 17
8	Tracce di esercitazione **a tempo** (2h30 su carta), orale	18, 19, 20

IL SEGRETO PER PASSARE

Lo scritto premia chi ha **scritto a mano tante funzioni sulle liste**. Leggere le soluzioni non basta: prendi una traccia, chiudi il fascicolo, scrivi su carta in 2h30, poi correggi confrontando con le soluzioni e provando il codice al computer (Dev-C++, come al corso).

CAPITOLO 1

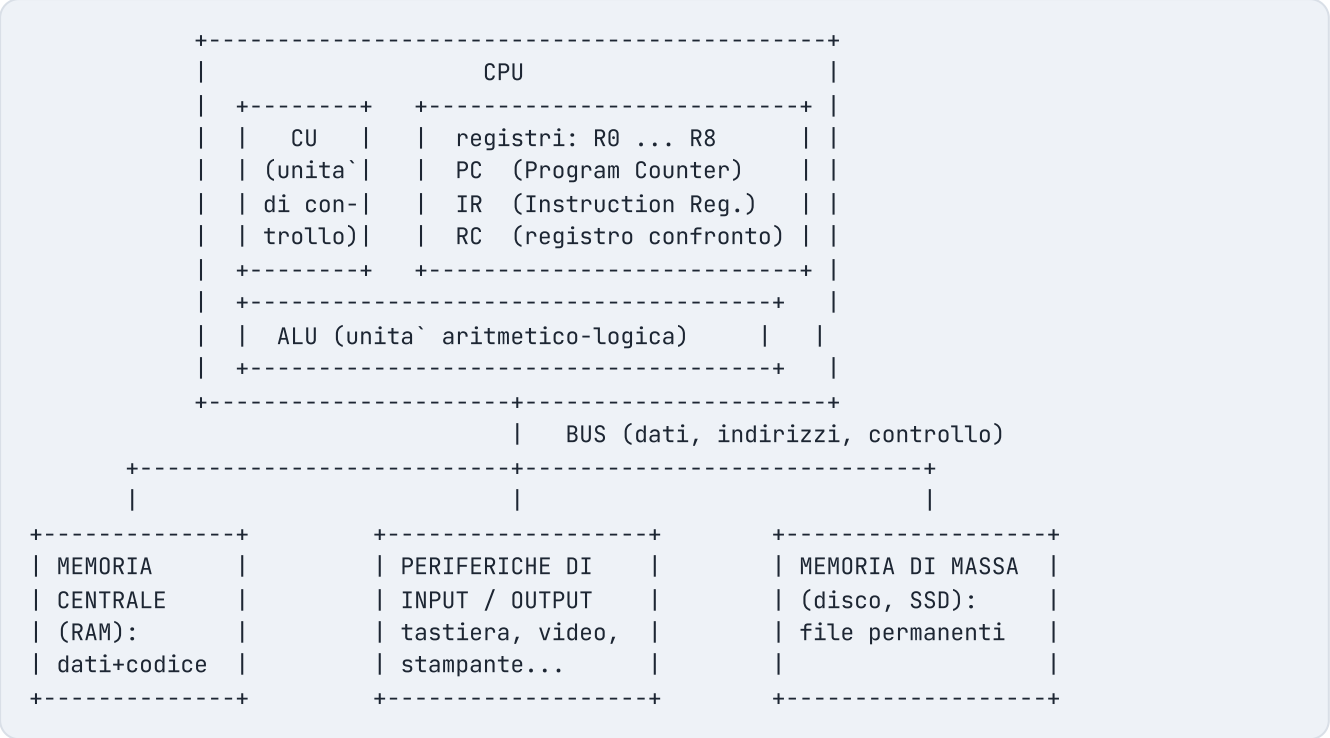
Il sistema di elaborazione

(slide lezioni 1-3 e "Cenni sul linguaggio macchina"; testo1 cap. 1, 2, 4.1-4.4)

Prima di programmare bisogna sapere **che cosa** esegue i programmi. Questo capitolo è teoria che può uscire all'orale.

La macchina di Von Neumann

Quasi tutti i calcolatori seguono lo schema di **Von Neumann**: **i dati e il programma stanno nella stessa memoria**, e una CPU esegue le istruzioni una dopo l'altra.



- **CPU** (Central Processing Unit), il "cervello":
- **CU** (Control Unit): legge le istruzioni, le **decodifica** e comanda le altre parti;
- **ALU** (Arithmetic Logic Unit): fa i calcoli (somme, confronti, operazioni logiche);
- **registri**: piccole memorie velocissime dentro la CPU. I più importanti: **PC** (*Program Counter*: contiene l'**indirizzo** della prossima istruzione) e **IR** (*Instruction Register*: contiene l'istruzione in esecuzione).
- **Memoria centrale (RAM)**: una lunga sequenza di **locazioni**, ognuna con un **indirizzo** (un numero). È **volatile**: si svuota quando spegni.
- **Memoria di massa** (disco, SSD): lenta ma **permanente**; contiene i file.
- **Periferiche**: input (tastiera, mouse), output (video, stampante).
- **Bus**: i "fili" che collegano tutto (trasportano dati, indirizzi e segnali di controllo).

Il ciclo Fetch - Decode - Execute

La CPU ripete all'infinito tre passi:

1. **Fetch** (prelievo): legge dalla RAM l'istruzione all'indirizzo scritto nel **PC** e la copia in **IR**; poi il PC viene fatto avanzare all'istruzione successiva.
2. **Decode** (decodifica): la CU capisce che istruzione è (dal suo **codice operativo**) e attiva il circuito giusto.
3. **Execute** (esecuzione): l'istruzione viene eseguita (un calcolo nella ALU, un trasferimento RAM-registro, un salto...).

Un **salto** (*jump*) non fa altro che **scrivere un nuovo indirizzo nel PC**: così la prossima istruzione prelevata non è quella "dopo", ma quella indicata.

Il linguaggio macchina del corso

A lezione si usa una macchina semplificata con una dozzina di istruzioni. Ogni istruzione è una parola di 32 bit: **8 bit di codice operativo**, poi gli operandi.

Istruzione	Codice	Formato	Cosa fa
LOAD	`00000000`	codice, registro (4 bit), indirizzo (20 bit)	copia nel registro il contenuto della locazione
STORE	`00000001`	codice, registro, indirizzo	copia nella locazione il contenuto del registro
ADD	`00000010`	codice, Ri, Rj	Ri = Ri + Rj
SUB	`00000100`	codice, Ri, Rj	Ri = Ri - Rj
MULT	`00000110`	codice, Ri, Rj	Ri = Ri * Rj
DIV	`00001000`	codice, Ri, Rj	Ri = Ri / Rj
MOD	`00001010`	codice, Ri, Rj	resto della divisione
READ	`00010000`	codice, unità (4 bit), indirizzo	legge un dato da input nella locazione
WRITE	`00010001`	codice, unità, indirizzo	manda in output il dato della locazione
COMP	`00100000`	codice, Ri, Rj	in RC: -1 se RiRj
JMP (salto condizionato)	`00100001`	codice, Ri, Rj, indirizzo	se Ri == Rj mette l'indirizzo nel PC
STOP	`10000001`	codice	termina il programma

Esempio della lezione: sommare le parole di indirizzo 64 e 68 e mettere il risultato all'indirizzo 60.

```
00000000 0000 0000000000001000000   LOAD  R0 <- RAM[64]
00000000 0001 0000000000001000100   LOAD  R1 <- RAM[68]
00000010 0000 0001                  ADD   R0 <- R0 + R1
00000001 0000 0000000000000111100   STORE RAM[60] <- R0
10000001                            STOP
```

Per eseguirlo ("RUN") si mette nel PC l'indirizzo della prima istruzione; da lì parte il ciclo fetch-decode-execute.

Il software di base

- **Sistema operativo (SO)**: il programma che gestisce la macchina e la mette a disposizione degli altri programmi: gestisce i processi (programmi in esecuzione), la memoria, le periferiche e il **file system**.
- **File system**: organizza i dati della memoria di massa in **file** e **cartelle** (directory) in una struttura ad albero. Un file si identifica con il suo **percorso** (es. `C:\esercizi\voli.c`).

Dal programma C al programma eseguibile

Il processore capisce solo il linguaggio macchina. Un programma scritto in un **linguaggio ad alto livello** come il C va **tradotto**:

```
voli.c  --[preprocessore]-->  (testo con #include e #define espansi)
        --[compilatore]--->   voli.o   (codice oggetto, linguaggio macchina)
        --[linker]-------->   voli.exe (eseguibile: unito alle librerie, es. printf)
        --[loader del SO]->   caricato in RAM ed eseguito
```

- Il **preprocessore** gestisce le righe che iniziano con `#` (`#include <stdio.h>`, `#define N 10`).
- Il **compilatore** controlla il programma e lo traduce. Se trova errori **non produce** l'eseguibile.
- Il **linker** collega il tuo codice con quello delle librerie (ad esempio il codice di `printf`).
- Un **IDE** come **Dev-C++** (usato al corso) fa tutto con un clic e include un **debugger**.

DOMANDE D'ORALE TIPICHE

"Descrivi l'architettura di Von Neumann." "Cosa contengono PC e IR?" "Cosa succede nel ciclo fetch-decode-execute?" "Che differenza c'è tra compilatore e linker?" "Cosa fa un'istruzione di salto?" Rispondi con il disegno e con un esempio.

ESERCIZIO 1.1

Scrivi in linguaggio macchina (con i codici della tabella) un programma che legge due numeri dall'unità di input `0000` nelle locazioni 100 e 104, li moltiplica e scrive il risultato nella locazione 108, poi lo manda all'unità di output `0001`.

Soluzione

```
00010000 0000 <ind. 100>    READ  RAM[100] <- input
00010000 0000 <ind. 104>    READ  RAM[104] <- input
00000000 0000 <ind. 100>    LOAD  R0 <- RAM[100]
00000000 0001 <ind. 104>    LOAD  R1 <- RAM[104]
00000110 0000 0001          MULT  R0 <- R0 * R1
00000001 0000 <ind. 108>    STORE RAM[108] <- R0
00010001 0001 <ind. 108>    WRITE output <- RAM[108]
10000001                    STOP
```

Gli indirizzi in binario su 20 bit sono: 100 = `00000000000001100100`, 104 = `00000000000001101000`, 108 = `00000000000001101100`.

CAPITOLO 2

Rappresentazione dell'informazione

(slide lezione 7, complementi "Sistemi di numerazione" e "Rappresentazione di numeri"; testo1 app. D)

Nella memoria ci sono solo **bit** (0 e 1). Questo capitolo spiega come con i bit si rappresentano numeri interi, negativi e reali. Il **floating point** è uscito come **esercizio 3** in un testo d'esame: va saputo fare a mano, senza errori.

Numeri e numerali

Un **numero** è un concetto; un **numerale** è la scrittura che lo rappresenta. Il numero dodici si scrive `12` in decimale, `XII` in cifre romane, `1100` in binario, `C` in esadecimale.

Nei **sistemi posizionali** ogni cifra "pesa" a seconda della posizione. In base b, il numerale $c_{n-1}...c_1c_0$ vale:

> $c_{n-1}\cdot b^{n-1} + ... + c_1\cdot b^1 + c_0\cdot b^0$

Base	Cifre	Esempio
2 (binario)	0, 1	`10011100`$_2$ = 128+16+8+4 = 156
8 (ottale)	0...7	`234`$_8$ = 2·64+3·8+4 = 156
10 (decimale)	0...9	156
16 (esadecimale)	0...9, A...F	`9C`$_{16}$ = 9·16+12 = 156

Conversioni

Da base 10 a base 2 (parte intera): divisioni successive

Dividi per 2, scrivi il resto, continua col quoziente finché arrivi a 0. Il numero binario sono i **resti letti dal basso verso l'alto**.

```
156 : 2 = 78  resto 0   ^
 78 : 2 = 39  resto 0   |
 39 : 2 = 19  resto 1   |   leggo dal basso:
 19 : 2 =  9  resto 1   |   156 = 10011100 (base 2)
  9 : 2 =  4  resto 1   |
  4 : 2 =  2  resto 0   |
  2 : 2 =  1  resto 0   |
  1 : 2 =  0  resto 1   |
```

Da base 2 a base 10: somma dei pesi

`10011100` $_2$: i pesi delle cifre 1 sono 128, 16, 8, 4, quindi 128 + 16 + 8 + 4 = **156**.

Base 2, 8 e 16: a gruppi di bit

- in **ottale**, raggruppa i bit **a 3 a 3** partendo da destra: `10 011 100` = `2 3 4` $_8$;
- in **esadecimale**, raggruppa i bit **a 4 a 4**: `1001 1100` = `9 C` $_{16}$.

Parte frazionaria: moltiplicazioni successive

Moltiplica la parte frazionaria per 2: la parte intera del risultato (0 o 1) è la prossima cifra; continua con la nuova parte frazionaria. Si legge **dall'alto verso il basso**.

```
0.625 x 2 = 1.25  -> 1      leggo dall'alto:
0.25  x 2 = 0.5   -> 0      0.625 = 0.101 (base 2)
0.5   x 2 = 1.0   -> 1      (finisce: la parte frazionaria e` 0)
```

Molti numeri decimali "semplici" in binario sono **periodici** (infiniti): 0.1 = 0.000110011001100... Per questo `float` e `double` rappresentano molti reali **solo in modo approssimato**.

Interi senza segno: binario puro

Con **n bit** si hanno 2^n numerali, quindi si rappresentano i numeri da **0 a 2^n - 1** (con 8 bit: da 0 a 255).

Somma binaria: come in decimale, con i riporti. 0+0=0, 0+1=1, 1+1=0 con riporto 1, 1+1+1=1 con riporto 1.

```
riporti:  1111111
            0001111001   (121)
          + 1001001111   (591)
          ------------
            1011001000   (712)
```

Se il risultato non sta negli n bit si ha **overflow**.

Interi con segno

Modulo e segno

Il primo bit è il segno (0 = +, 1 = -), gli altri n-1 bit sono il valore assoluto. Intervallo: $[-(2^{n-1}-1), 2^{n-1}-1]$. Difetti: **due zeri** (`000` e `100` con 3 bit) e circuiti più complicati per i calcoli. Per questo non si usa.

Complemento a due (quello vero)

Con n bit:

- se V ≥ 0: la rappresentazione è il binario puro di V su n bit;
- se V < 0: la rappresentazione è il binario puro di **$2^n + V$** (cioè 2^n - |V|).

Intervallo rappresentabile: **$[-2^{n-1}, 2^{n-1} - 1]$**. Con 8 bit: da -128 a +127. C'è un solo zero. Il primo bit vale 1 per tutti i negativi.

```
n = 3:   000  001  010  011  100  101  110  111
          0    1    2    3   -4   -3   -2   -1
```

Metodo pratico per un negativo: scrivi |V| in binario su n bit, **inverti tutti i bit** (complemento a uno), **aggiungi 1**.

```
-591 con 11 bit:   591       = 01001001111
                   invertito = 10110110000
                   +1        = 10110110001
verifica: 2^11 - 591 = 2048 - 591 = 1457 = 10110110001  ok
```

Estensione del segno: per scrivere lo stesso numero con più bit, si ripete a sinistra il primo bit. -1 è `111` su 3 bit, `1111` su 4, `11111111` su 8.

Sottrazione = somma del negativo: N - M = N + (-M), si sommano le rappresentazioni e si scarta l'eventuale riporto oltre l'ultimo bit.

Overflow in complemento a due: sommando due positivi si ottiene un "negativo" (o viceversa). Su 8 bit: 120 + 10 = 130, che non sta in [-128, 127]: `01111000 + 00001010 = 10000010`, che si legge -126.

ESERCIZIO D'ESAME RISOLTO (DAL COMPLEMENTO DIDATTICO DEL PROFESSORE)

N = 121, M = 591. (1) Numero minimo B di bit per scrivere N e M in binario puro. (2) Scriverli su B bit e calcolare N+M. (3) Numero minimo di bit per scrivere N e -M in complemento a due. (4) Calcolare N - M su 16 bit.

Soluzione

1. 121 = `1111001` (7 bit); 591 = `1001001111` (10 bit). Quindi **B = 10**.
2. N = `0001111001`, M = `1001001111`; la somma (vedi sopra) è `1011001000` = 512+128+64+8 = **712**. Ok.
3. N = 121 sta in $[-2^7, 2^7-1]$ = [-128, 127], quindi gli bastano 8 bit; -591 richiede [-1024, 1023], cioè **11 bit**. Con 11 bit: N = `00001111001`, -M = `10110110001`.
4. Su 16 bit: N = `0000000001111001`, -M = `1111110110110001` (estensione del segno con 1). Somma: `1111111000101010`, che è -470 = 121 - 591. Verifica: 2^{16} - 470 = 65066 = `1111111000101010`. Ok.

Numeri reali: il floating point a 32 bit

Viene dalla **notazione scientifica**: 223365000 = 2.23365×10^8, con una **mantissa** (le cifre) e un **esponente** (l'ordine di grandezza). In binario si **normalizza** la mantissa nella forma **1.xxxx**.

Il `float` occupa **32 bit**:

```
| s |    E (8 bit)    |               M (23 bit)                |
  1   esponente in       cifre della mantissa normalizzata
segno eccesso 127        DOPO l'"1." (l'1 iniziale non si scrive)
```

Il numero rappresentato è: $\mathbf{(-1)^s \times 1.M \times 2^{E-127}}$

- **s**: 0 positivo, 1 negativo;
- **E**: l'esponente vero **più 127** (si dice "in eccesso 127"). Così E va da 0 a 255 e l'esponente vero può essere negativo;
- **M**: le cifre dopo la virgola della mantissa normalizzata, completate con zeri a destra fino a 23 bit.

Procedura in 5 passi (da usare all'esame)

1. **Segno**: s = 0 se positivo, 1 se negativo. Da qui lavora col valore assoluto.
2. **Converti in binario** parte intera (divisioni) e parte frazionaria (moltiplicazioni).
3. **Normalizza**: sposta la virgola subito dopo il primo 1. Conta di quante posizioni l'hai spostata: è l'esponente **e** (positivo se l'hai spostata a sinistra, negativo se a destra).
4. **Esponente**: E = e + 127, scritto in binario su 8 bit.
5. **Mantissa**: M = le cifre dopo "1.", portate a 23 bit (zeri a destra se mancano; se avanzano, si **troncano**).

Esempio: 4.25 (dalle slide)

4.25 = `100.01`$_2$ → normalizzato `1.0001` $\times 2^2$ → E = 2 + 127 = 129 = `10000001`, M = `0001`.

```
4.25  =  0 10000001 00010000000000000000000
```

Esempio: 796.25 (dalle slide)

796 = `1100011100`, 0.25 = `.01` → `1100011100.01` = `1.10001110001` $\times 2^9$. E = 9 + 127 = 136 = `10001000`; M = `10001110001` + zeri.

```
796.25 = 0 10001000 10001110001000000000000
```

Esempio inverso: da bit a numero

`1 10000000 10010010000000000000000`: s = 1 (negativo); E = 128, quindi e = 128 - 127 = 1; mantissa = 1.1001001 = 1 + 1/2 + 1/16 + 1/128 = 1.5703125. Numero = -1.5703125 $\times 2^1$ = **-3.140625**.

ESERCIZIO 3 D'ESAME: FLOATING POINT DI 1216.37

"Realizzare la rappresentazione in floating point su 32 bit del numero razionale 1216.37. Mostrare e descrivere accuratamente i passaggi."

Soluzione

1. Segno: positivo, s = `0`.

2a. Parte intera 1216 (divisioni per 2):

```
1216:2=608 r0   608:2=304 r0   304:2=152 r0   152:2=76 r0
  76:2= 38 r0    38:2= 19 r0    19:2=  9 r1     9:2= 4 r1
   4:2=  2 r0     2:2=  1 r0     1:2=  0 r1
letti dal basso:  1216 = 10011000000   (11 cifre; verifica: 1024+128+64 = 1216)
```

2b. Parte frazionaria 0.37 (moltiplicazioni per 2). Ci servono abbastanza cifre: la mantissa ha 23 bit, 10 vanno alla parte intera (le cifre dopo il primo 1), quindi ne servono **13** della parte frazionaria (più una per controllo):

```
0.37x2=0.74 ->0   0.74x2=1.48 ->1   0.48x2=0.96 ->0   0.96x2=1.92 ->1
0.92x2=1.84 ->1   0.84x2=1.68 ->1   0.68x2=1.36 ->1   0.36x2=0.72 ->0
0.72x2=1.44 ->1   0.44x2=0.88 ->0   0.88x2=1.76 ->1   0.76x2=1.52 ->1
0.52x2=1.04 ->1   0.04x2=0.08 ->0   ...  (non finisce mai: e` periodico)
0.37 = 0.0101111010111 0...  (base 2)
```

Quindi 1216.37 ≈ `10011000000.0101111010111`

3. Normalizzo: sposto la virgola di **10** posizioni a sinistra, subito dopo il primo 1:
`1.00110000000101111010111` × 2^{10}

4. Esponente: E = 10 + 127 = 137 = `10001001`

5. Mantissa: le 23 cifre dopo "1.": `0011000000` (10 cifre dalla parte intera) + `0101111010111` (13 cifre dalla frazione) = `00110000000101111010111`

```
1216.37  =  0 10001001 00110000000101111010111
```

Osservazione da scrivere: la parte frazionaria è periodica, quindi la rappresentazione è **approssimata**; le cifre oltre la 23esima vanno perse (il valore memorizzato è circa 1216.36999512). La cifra successiva era 0, quindi troncare o arrotondare dà lo stesso risultato. Verifica: `1.0011` × 2^{10} = `10011000000` = 1216. Ok.

ESERCIZIO 2.2

Rappresenta in floating point a 32 bit: (a) 5.375, (b) -45.625. Poi (c) calcola il numero rappresentato da `1 10000100 01101101000000000000000`.

Soluzione

(a) 5.375 = `101.011` = `1.01011` × 2^2; E = 129 = `10000001` → `0 10000001 01011000000000000000000`.

(b) 45 = `101101`, 0.625 = `.101` → `101101.101` = `1.01101101` × 2^5; s = 1, E = 132 = `10000100` → `1 10000100 01101101000000000000000`.

(c) È proprio il risultato di (b): s = 1, E = 132 → e = 5, mantissa `1.01101101`, quindi `-101101.101` = **-45.625**.

Caratteri

Anche i caratteri sono numeri: ogni carattere ha un **codice** nella tabella **ASCII** (7 bit, di solito in un byte). In C il tipo `char` è un piccolo intero.

Carattere	Codice	Carattere	Codice
`'0'` … `'9'`	48 … 57	`'a'` … `'z'`	97 … 122
`'A'` … `'Z'`	65 … 90	`' '` (spazio)	32
`'\0'` (fine stringa)	0	`'\n'` (a capo)	10

Conseguenze utili: `'7' - '0'` vale 7 (da carattere cifra a numero); `'a' - 'A'` vale 32 (da maiuscola a minuscola basta sommare 32); le lettere sono consecutive, quindi `c >= 'a' && c <= 'z'` controlla se `c` è minuscola.

ALL'ORALE

Sappi spiegare **perché** si usa il complemento a due (un solo zero, la sottrazione diventa una somma), **perché** i float sono approssimati (numero finito di bit, frazioni periodiche) e perché l'esponente è in eccesso 127 (per avere esponenti negativi senza un bit di segno in più).

CAPITOLO 3

Primi passi in C: algoritmi, variabili, tipi, espressioni

(slide lezioni 3-5; complementi lezioni 4 e 5; testo1 cap. 3, 5, 6, 8, app. B e C)

Algoritmo e programma

Un **algoritmo** è una sequenza **finita** di passi **precisi** (non ambigui) che risolve un problema in un tempo finito. Un **programma** è un algoritmo scritto in un linguaggio di programmazione. Prima si pensa l'algoritmo (anche con un diagramma a blocchi o in italiano), poi lo si traduce in C.

La **programmazione strutturata** (teorema di Böhm-Jacopini) dice che ogni algoritmo si può scrivere usando solo tre strutture: **sequenza**, **selezione** (`if`) e **iterazione** (`while`). Niente salti `goto` .

Il primo programma

```c
#include <stdio.h>              /* libreria per input/output (printf, scanf) */

int main(void) {                /* qui comincia l'esecuzione                 */
  int lato, area;               /* dichiarazione delle variabili             */

  printf("Lato del quadrato? ");
  scanf("%d", &lato);           /* & = "indirizzo di": scanf scrive in lato  */
  area = lato * lato;
  printf("Area = %d\n", area);
  return 0;                     /* 0 = "tutto bene" al sistema operativo     */
}
```

- Ogni istruzione finisce con `;` . I blocchi stanno tra `{ }` .
- I **commenti** in ANSI C si scrivono `/* ... */` (il `//` non è ANSI C89).
- In ANSI C le **dichiarazioni** stanno **all'inizio del blocco**, prima delle istruzioni.

Variabili e tipi semplici

Una **variabile** è una locazione di memoria con un **nome**, un **tipo** e un **valore**. Il tipo dice quanti byte occupa e come interpretare i bit.

Tipo	Cosa contiene	Dimensione tipica	Formato printf/scanf
`char`	un carattere (piccolo intero)	1 byte	`%c`
`int`	intero	4 byte	`%d`
`long`	intero più grande	4 o 8 byte	`%ld`
`float`	reale, precisione singola	4 byte	`%f`
`double`	reale, precisione doppia	8 byte	`%f` in printf, `%lf` **in scanf**

`sizeof(tipo)` dà la dimensione in byte (servirà con `malloc`).

Costanti: `#define MAX 100` (il preprocessore sostituisce `MAX` con `100` prima della compilazione; per convenzione in MAIUSCOLO). Costanti carattere tra apici `'A'`, stringhe tra virgolette `"ciao"`.

VARIABILI NON INIZIALIZZATE

Una variabile locale appena dichiarata contiene un valore **casuale** ("spazzatura"). `int somma; somma = somma + x;` è sbagliato: scrivi `int somma = 0;`. Vale per contatori, somme e per i puntatori (`TipoLista l = NULL;`).

Input e output formattato

```
printf("x vale %d, y vale %.2f\n", x, y);  /* %.2f: due cifre decimali       */
scanf("%d %lf", &n, &d);                   /* & davanti alle variabili!      */
scanf("%s", parola);                       /* per le stringhe NIENTE &       */
```

Formati utili: `%5d` (larghezza 5), `%-10s` (allineato a sinistra), `%%` (il simbolo %). `\n` va a capo, `\t` tabulazione.

`scanf` **restituisce** quanti valori è riuscita a leggere: si usa per controllare l'input e, con i file, per sapere quando i dati finiscono.

Espressioni e operatori

Categoria	Operatori	Note
aritmetici	`+ - * / %`	`%` (resto) solo tra interi
relazionali	`< <= > >= == !=`	danno 1 (vero) o 0 (falso)
logici	`&& || !`	AND, OR, NOT
assegnamento	`= += -= *= /= %=`	`x += 3` equivale a `x = x + 3`
incremento	`++ --`	prefisso o postfisso
condizionale	`c ? a : b`	vale `a` se `c` è vera, sennò `b`

Divisione intera: attenzione

codice/basi.c

```c
printf("%d %d %d\n", 7 / 2, 7 % 2, -7 / 2);   /* 3 1 -3              */
printf("%.2f %.2f\n", 7 / 2.0, (double) 7 / 2); /* 3.50 3.50          */
```

Se **entrambi** gli operandi sono interi, `/` fa la **divisione intera** (tronca verso lo zero). Per avere il risultato reale almeno un operando deve essere reale: `7 / 2.0` oppure il **cast** `(double) 7 / 2`.

Conversioni di tipo

- **Implicite**: in un'espressione mista, il tipo "più piccolo" viene promosso (`int + double` diventa `double`). In un assegnamento il valore viene convertito nel tipo della variabile: `int i = 8.7;` mette **8** in `i` (tronca).
- **Esplicite** (cast): `(tipo) espressione`.

Pre e post incremento

```c
int i = 5, a, b;
a = i++;     /* a = 5, poi i diventa 6 (POST: prima usa, poi incrementa) */
b = ++i;     /* i diventa 7, poi b = 7 (PRE: prima incrementa, poi usa)  */
```

Precedenza (dalla più alta)

`!` `++` `--` `(cast)` → `*` `/` `%` → `+` `-` → `<` `<=` `>` `>=` → `==` `!=` → `&&` → `||` → `?:` → `=` `+=` ... Nel dubbio, **usa le parentesi**.

Algebra di Boole ed espressioni logiche

In C **0 è falso** e **qualunque valore diverso da 0 è vero**.

a	b	a && b	a || b	!a
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0

Leggi di De Morgan (servono per negare le condizioni dei cicli):

- `!(a && b)` equivale a `!a || !b`
- `!(a || b)` equivale a `!a && !b`

Esempio: il contrario di "x è tra 1 e 10" `(x >= 1 && x <= 10)` è `(x < 1 || x > 10)`.

Valutazione cortocircuitata: in `a && b`, se `a` è falsa, `b` **non viene valutata**; in `a || b`, se `a` è vera, `b` non viene valutata. È fondamentale con le liste:

```
while (p != NULL && p->info != x)   /* se p e` NULL, p->info NON viene valutato */
```

Scambiando l'ordine (`p->info != x && p != NULL`) il programma va in crash quando `p` è `NULL`.

= INVECE DI ==

`if (x = 0)` **assegna** 0 a x e la condizione è sempre falsa. Per confrontare si usa `==`. Altro classico: `if (0 < x < 10)` non fa quello che sembra; si scrive `if (0 < x && x < 10)`.

ESERCIZIO 3.1 (ESECUZIONE SIMULATA)

Cosa stampa questo frammento?

```
int a = 7, b = 2, c;
double d;
c = a / b + a % b;
d = a / b;
printf("%d %.1f %d\n", c, d, a > b && b > 3 || !0);
```

Soluzione

`a / b` = 3 (divisione intera), `a % b` = 1, quindi c = **4**. `d = a / b` calcola prima 3 (intero) e poi lo converte: d = **3.0**. L'ultima espressione: `a > b` è 1, `b > 3` è 0, quindi `1 && 0` = 0; poi `0 || !0` = `0 || 1` = **1** (`&&` ha precedenza su `||`). Stampa: `4 3.0 1`.

CAPITOLO 4

Istruzioni di controllo

(slide lezioni 4-6, EG1-EG3; testo1 cap. 4, 5, 7, 9)

Selezione: if / else

```
if (voto >= 18)
  printf("promosso\n");
else
  printf("respinto\n");
```

Se in un ramo ci sono più istruzioni servono le graffe `{ }`. Con più casi si fanno catene `else if`:

```
if (temp < 0)
  printf("gelo\n");
else if (temp < 20)
  printf("fresco\n");
else
  printf("caldo\n");
```

L'ELSE "AMBIGUO" (DANGLING ELSE)

Un `else` si attacca sempre all'`if` **più vicino** senza `else`, qualunque sia l'indentazione:

```
if (a > 0)
  if (b > 0)
    printf("entrambi positivi\n");
else                        /* appartiene a  if (b > 0) , NON a  if (a > 0) */
  printf("a non positivo?\n");
```

Per attaccarlo al primo `if` servono le graffe: `if (a > 0) { if (b > 0) ... } else ...`. È un esercizio delle EG (`ambiguif.c`).

Selezione multipla: switch

```c
switch (scelta) {              /* scelta deve essere int o char      */
  case 1:
    stampaTabella(t);
    break;                     /* senza break si "cade" nel case dopo */
  case 2:
  case 3:                      /* 2 e 3 fanno la stessa cosa          */
    aggiungi(&t);
    break;
  default:                     /* tutti gli altri valori              */
    printf("opzione sbagliata\n");
}
```

Lo `switch` è tipico dei **menu** (vedi il programma dei voli nelle EG).

Iterazione: while, do-while, for

```c
/* while: controlla PRIMA; il corpo puo` non essere mai eseguito */
while (condizione) {
  ...
}

/* do-while: controlla DOPO; il corpo e` eseguito ALMENO una volta */
do {
  printf("numero positivo? ");
  scanf("%d", &n);
} while (n <= 0);

/* for: comodo quando si conosce il numero di ripetizioni */
for (i = 0; i < n; i++) {
  ...
}
```

Il `for (inizializzazione; condizione; aggiornamento)` equivale a:

```c
inizializzazione;
while (condizione) {
  corpo;
  aggiornamento;
}
```

`break` esce subito dal ciclo; `continue` salta al prossimo giro. Usali poco: rendono i cicli più difficili da leggere.

Gli schemi di ciclo da sapere a memoria

Quasi tutti gli esercizi sono combinazioni di questi schemi. All'esame, sulle liste, li userai identici.

Contatore: quanti elementi soddisfano una condizione.

```
conta = 0;
for (i = 0; i < n; i++)
  if (v[i] > soglia)
    conta++;
```

Accumulatore: somma (o prodotto) di valori.

```
somma = 0;                    /* per un prodotto: prod = 1 */
for (i = 0; i < n; i++)
  somma += v[i];
media = (double) somma / n;    /* attenzione: n > 0 ! */
```

Massimo / minimo: si parte dal **primo elemento**, non da 0 (se tutti i valori fossero negativi, 0 sarebbe sbagliato).

```
max = v[0];
for (i = 1; i < n; i++)
  if (v[i] > max)
    max = v[i];
```

Sentinella: si leggono dati finché arriva un valore speciale.

```
scanf("%d", &x);
while (x != 0) {              /* 0 = sentinella: fine dei dati */
  ...elabora x...
  scanf("%d", &x);            /* lettura del prossimo */
}
```

Ricerca con variabile flag (stile del corso): ci si ferma appena si trova.

```
trovato = 0;
i = 0;
while (!trovato && i < n)
  if (v[i] == x)
    trovato = 1;
  else
    i++;
/* qui: se trovato, i e` la posizione */
```

Esecuzione simulata

All'esame e all'orale ti può essere chiesto di "eseguire a mano" un programma. Si fa una **tabella**: una colonna per variabile, una riga per ogni passo in cui qualcosa cambia.

```c
int i, s = 0;
for (i = 1; i <= 4; i++)
  if (i % 2 == 0)
    s = s + i;
printf("%d\n", s);
```

passo	i	i % 2 == 0 ?	s
inizio	-	-	0
1	1	no	0
2	2	sì	2
3	3	no	2
4	4	sì	6
fine	5 (esce)	-	6 → stampa **6**

ESERCIZIO 4.1

Scrivi un programma che legge una sequenza di interi terminata da 0 e stampa quanti sono positivi, quanti negativi e la media dei positivi (con due decimali).

Soluzione

```c
#include <stdio.h>

int main(void) {
  int x, pos = 0, neg = 0, sommaPos = 0;

  printf("numero (0 per finire): ");
  scanf("%d", &x);
  while (x != 0) {
    if (x > 0) {
      pos++;
      sommaPos += x;
    } else
      neg++;
    printf("numero (0 per finire): ");
    scanf("%d", &x);
  }
  printf("positivi %d, negativi %d\n", pos, neg);
  if (pos > 0)
    printf("media positivi %.2f\n", (double) sommaPos / pos);
  return 0;
}
```

Nota il controllo `pos > 0` prima della divisione e il cast a `double`.

ESERCIZIO 4.2

Scrivi un ciclo che stampa i numeri primi minori di 100 (come `primiCento.c` delle EG).

Soluzione

```c
int n, d, primo;
for (n = 2; n < 100; n++) {
  primo = 1;                          /* flag: suppongo che sia primo */
  for (d = 2; d * d <= n && primo; d++)
    if (n % d == 0)
      primo = 0;                      /* trovato un divisore          */
  if (primo)
    printf("%d ", n);
}
```

Basta provare i divisori fino a $\sqrt{n}$ (`d * d <= n`) e fermarsi al primo divisore trovato.

CAPITOLO 5

Funzioni e record di attivazione

(slide lezioni 8 e 10, complemento "Record di attivazione"; testo1 cap. 11)

Perché le funzioni

Una funzione è un **sottoprogramma** con un nome, che risolve un sottoproblema. Serve a non ripetere codice, a dividere un problema grande in pezzi (**modularizzazione**, progettazione *top-down*) e a provare ogni pezzo separatamente.

Dichiarazione, definizione, chiamata

```c
#include <stdio.h>

int mcd(int, int);                /* 1. DICHIARAZIONE (prototipo): prima del main */

int main(void) {
  int a = 15, b = 5, m;
  m = mcd(a, b);                  /* 2. CHIAMATA: a e b sono i parametri ATTUALI */
  printf("%d\n", m);
  return 0;
}
```

codice/basi.c

```c
int mcd(int n, int m) {           /* algoritmo di Euclide per sottrazioni */
  while (n != m)
    if (m > n)
      m = m - n;
    else
      n = n - m;
  return n;
}
```

- **Prototipo**: tipo restituito, nome, tipi dei parametri. Permette al compilatore di controllare le chiamate.
- **Definizione**: intestazione + corpo. `n` e `m` sono i **parametri formali**.
- **Chiamata**: i parametri **attuali** sono **espressioni**: il loro **valore** viene **copiato** nei parametri formali.
- `return espressione;` termina la funzione e restituisce il valore al chiamante. Una funzione `void` non restituisce nulla.

Passaggio dei parametri per valore

In C **i parametri passano sempre per valore**: la funzione riceve una **copia**. Modificare un parametro formale **non cambia** la variabile del chiamante.

codice/basi.c

```c
void scambioNO(int a, int b) {     /* SBAGLIATA: scambia due COPIE      */
  int t = a;
  a = b;
  b = t;
}

void scambio(int *pa, int *pb) {   /* GIUSTA: lavora sugli originali    */
  int t = *pa;
  *pa = *pb;
  *pb = t;
}
```

`scambioNO(a, b)` scambia solo le copie: dopo la chiamata `a` e `b` sono uguali a prima. Per modificare una variabile del chiamante bisogna passarne l'**indirizzo** (`scambio(&a, &b)`): lo vedremo bene con i puntatori (capitolo 8). È il meccanismo dei **parametri di output**, che userai in ogni esercizio 1 d'esame.

Il record di attivazione (RDA)

Ogni volta che una funzione viene **chiamata** succede questo:

1. l'esecuzione del chiamante si **sospende**;
2. viene allocata in memoria (sulla **pila**, *stack*) una zona chiamata **record di attivazione** (RDA). Contiene: - i **parametri formali**, riempiti con le copie dei valori dei parametri attuali; - le **variabili locali**; - il **valore di ritorno** e l'**indirizzo di ritorno** (dove riprendere nel chiamante);
3. si esegue il corpo della funzione, usando le locazioni del suo RDA;
4. con `return` il valore torna al chiamante, l'RDA viene **deallocato** (sparisce) e il chiamante riprende.

```
chiamata m = mcd(15, 5) dentro main:

        PILA (cresce verso l'alto)
   +-------------------------------+
   | RDA mcd                       |  <- in cima: la funzione in esecuzione
   |   parametri:  n = 15  m = 5   |
   |   ritorno a:  main, riga "m=" |
   +-------------------------------+
   | RDA main                      |
   |   variabili:  a=15 b=5 m=?    |
   +-------------------------------+

dopo il return: l'RDA di mcd sparisce, in main  m = 5
```

Le RDA si impilano: se `main` chiama `f` e `f` chiama `g` , sulla pila ci sono main, f, g; quando `g` finisce si torna a `f` . Questa pila diventa fondamentale per capire la **ricorsione** (capitolo 11).

Visibilità e ciclo di vita delle variabili

- **Variabili locali** (dichiarate in una funzione o in un blocco): **visibili solo lì**; **nascono** quando la funzione viene attivata e **muoiono** quando termina (vivono nell'RDA). Due funzioni possono avere variabili locali con lo stesso

nome: sono variabili diverse.

- **Variabili globali** (dichiarate fuori da ogni funzione): visibili in tutte le funzioni dopo la dichiarazione; vivono per tutto il programma. Da usare il meno possibile (rendono difficile capire chi modifica cosa).
- Variabili locali `static`: visibili solo nella funzione, ma **conservano il valore** tra una chiamata e l'altra.
- Una variabile locale **nasconde** una globale con lo stesso nome (*shadowing*).

```c
int contatore = 0;                /* globale                          */

void f(void) {
  static int chiamate = 0;        /* inizializzata una volta sola     */
  int locale = 0;                 /* ricreata a ogni chiamata         */
  chiamate++;
  locale++;
  contatore++;
  printf("%d %d %d\n", chiamate, locale, contatore);
}
/* chiamando f() tre volte stampa: 1 1 1, poi 2 1 2, poi 3 1 3 */
```

RESTITUIRE L'INDIRIZZO DI UNA VARIABILE LOCALE

```c
int *sbagliata(void) {
  int x = 5;
  return &x;        /* x muore con l'RDA: il puntatore restituito non vale piu` */
}
```

Per restituire memoria che "sopravvive" alla funzione si usa `malloc` (capitolo 9).

Progettare con le funzioni (il metodo del corso)

Nelle lezioni e nelle EG il professore procede sempre in **fasi**:

1. **ambiente di calcolo**: scrivo il `main` con le variabili e le **chiamate** delle funzioni che mi servono, come se esistessero già;
2. **prototipi**: scrivo le dichiarazioni (che cosa riceve, che cosa restituisce ogni funzione);
3. **algoritmo** di ogni funzione, in italiano;
4. **definizione** in C;
5. **test** (capitolo 13).

All'esame fai lo stesso per ogni `eserN`: prima l'intestazione (cosa riceve, cosa restituisce, quali sono i parametri di output), poi l'algoritmo in due righe di commento, poi il codice.

DOMANDE D'ORALE

"Che cos'è un record di attivazione e cosa contiene?" "Differenza tra parametro formale e attuale?" "Perché `scambioNO` non funziona?" "Cosa vuol dire che il passaggio è per valore?" "Differenza tra variabile locale, globale e static?"

ESERCIZIO 5.1

Scrivi una funzione `int potenza(int base, int esp)` che calcola $base^{esp}$ (esp ≥ 0) senza usare librerie, e disegna la pila delle RDA durante la chiamata `x = potenza(2, 3);` fatta dal `main`.

Soluzione

```
int potenza(int base, int esp) {
  int i, risultato = 1;
  for (i = 0; i < esp; i++)
    risultato *= base;
  return risultato;
}
```

```
   +---------------------------------------+
   | RDA potenza                           |
   |  par:  base = 2   esp = 3             |
   |  loc:  i = 0,1,2,3   risultato = 1,2,4,8 |
   +---------------------------------------+
   | RDA main:  x = ?                      |
   +---------------------------------------+
al return l'RDA di potenza sparisce e in main x = 8
```

CAPITOLO 6

Array

(slide lezioni 10 e 11, complementi lezioni 10 e 11, EG4; testo1 cap. 10)

Che cos'è un array

Un **array** è una sequenza di **N elementi dello stesso tipo**, in locazioni **consecutive** di memoria, a cui si accede con un **indice** da **0 a N-1**.

```
#define N 5
int v[N];                       /* 5 interi: v[0] ... v[4]          */
int w[N] = {4, -2, 9, 0, 9};    /* inizializzazione                 */
double temp[] = {1.5, 2.0};     /* dimensione dedotta: 2             */
```

```
indice:    0     1     2     3     4
        +-----+-----+-----+-----+-----+
   w    |  4  | -2  |  9  |  0  |  9  |
        +-----+-----+-----+-----+-----+
```

- La dimensione deve essere una **costante** (in ANSI C), per questo si usa `#define`.
- Gli array **non si copiano** con `=` e **non si confrontano** con `==`: si lavora elemento per elemento con un ciclo.

USCIRE DALL'ARRAY

`v[N]` **non esiste** (l'ultimo è `v[N-1]`). Il C **non controlla** gli indici: scrivere fuori dall'array rovina altre variabili o fa andare in crash il programma, senza messaggi. Controlla sempre la condizione dei cicli: `i < N`, non `i <= N`.

Array e funzioni

Quando passi un array a una funzione, **passi l'indirizzo del suo primo elemento** (non una copia!). Conseguenze:

- la funzione **può modificare** l'array originale;
- la funzione **non conosce la dimensione**: va passata come parametro.

codice/basi.c

```c
/* un array passato a una funzione e` l'indirizzo del suo primo elemento:
   la dimensione va passata a parte                                      */
int massimo(int v[], int dim) {
  int i, max = v[0];
  for (i = 1; i < dim; i++)
    if (v[i] > max)
      max = v[i];
  return max;
}

double media(int v[], int dim) {
  int i, somma = 0;
  for (i = 0; i < dim; i++)
    somma += v[i];
  return (double) somma / dim;     /* cast: altrimenti divisione intera! */
}

/* ricerca lineare con variabile "flag" (stile del corso):
   restituisce l'indice di x oppure -1                                   */
int indiceDi(int v[], int dim, int x) {
  int trovato = 0, i = 0;
  while (!trovato && i < dim)
    if (v[i] == x)
      trovato = 1;
    else
      i++;
  if (trovato)
    return i;
  return -1;
}
```

L'intestazione `int massimo(int v[], int dim)` è equivalente a `int massimo(int *v, int dim)` (capitolo 8).

Una funzione non può restituire due valori con `return`. Per minimo **e** massimo si usano **parametri di output**:

codice/basi.c

```c
/* due risultati: uno con return? No: entrambi con parametri di output */
void minMax(int v[], int dim, int *pmin, int *pmax) {
  int i;
  *pmin = *pmax = v[0];
  for (i = 1; i < dim; i++) {
    if (v[i] < *pmin) *pmin = v[i];
    if (v[i] > *pmax) *pmax = v[i];
  }
}
```

Si chiama così: `minMax(v, N, &mn, &mx);`

Array parzialmente riempito

Spesso non si sa quanti dati ci saranno. Si alloca un array "abbastanza grande" (il **sostegno**) e si tiene in una variabile **quanti elementi sono davvero usati**. Gli elementi validi sono sempre i primi, da `0` a `quanti-1`.

```
#define MAX 100
int v[MAX];
int quanti = 0;              /* all'inizio l'array e` vuoto */

/* aggiunta in fondo */
if (quanti < MAX) {
  v[quanti] = x;
  quanti++;
}

/* eliminazione dell'elemento di indice k: sposto indietro quelli dopo */
for (i = k; i < quanti - 1; i++)
  v[i] = v[i + 1];
quanti--;
```

Questa è proprio l'idea della **tabella** (capitolo 10): un array di struct più un intero `quanti`.

Matrici (array a due dimensioni)

```
#define R 3
#define C 4
int m[R][C];                 /* 3 righe, 4 colonne */

for (i = 0; i < R; i++)      /* scorro le righe            */
  for (j = 0; j < C; j++)    /* e dentro ogni riga le colonne */
    m[i][j] = i * j;
```

In memoria la matrice è memorizzata **per righe**, una riga dopo l'altra. Passandola a una funzione, il numero di **colonne** va scritto: `void stampaMat(int m[][C], int righe)`.

Somma di matrici: `s[i][j] = a[i][j] + b[i][j]`. **Prodotto** (A di dimensione R×K, B di dimensione K×C):

```
for (i = 0; i < R; i++)
  for (j = 0; j < C; j++) {
    p[i][j] = 0;
    for (k = 0; k < K; k++)
      p[i][j] += a[i][k] * b[k][j];
  }
```

ESERCIZIO 6.1

Scrivi una funzione `int condensa(int v[], int dim)` che elimina dall'array tutti gli zeri, compattando gli altri elementi all'inizio, e restituisce quanti elementi restano. Esempio: `{3,0,0,5,0,7}` diventa `{3,5,7}` e restituisce 3.

Soluzione

```c
int condensa(int v[], int dim) {
  int i, k = 0;               /* k = prossima posizione libera */
  for (i = 0; i < dim; i++)
    if (v[i] != 0) {
      v[k] = v[i];
      k++;
    }
  return k;
}
```

Un solo ciclo: è più efficiente che spostare tutto indietro a ogni zero trovato.

ESERCIZIO 6.2

Scrivi una funzione che riceve una matrice quadrata N×N e restituisce 1 se è simmetrica (`m[i][j] == m[j][i]` per ogni i, j), 0 altrimenti.

Soluzione

```c
int simmetrica(int m[][N]) {
  int i, j;
  for (i = 0; i < N; i++)
    for (j = i + 1; j < N; j++)       /* basta la parte sopra la diagonale */
      if (m[i][j] != m[j][i])
        return 0;                     /* trovato un controesempio          */
  return 1;
}
```

CAPITOLO 7

File testuali e binari

(lezione 9 con video, EG7; testo1 cap. 17)

Le variabili vivono in RAM e spariscono a fine programma. Per conservare i dati servono i **file**, che stanno nella memoria di massa.

Aprire e chiudere un file

```c
#include <stdio.h>

FILE *f;                          /* puntatore a una struttura che descrive il file */
f = fopen("dati.txt", "r");       /* nome del file, modo di apertura                */
if (f == NULL) {                  /* fopen restituisce NULL se fallisce             */
  printf("impossibile aprire il file\n");
  return 0;
}
...
fclose(f);                        /* SEMPRE: scarica i dati e libera la risorsa     */
```

Modo	Significato
`"r"`	lettura; il file deve esistere
`"w"`	scrittura; crea il file o lo **svuota** se esiste
`"a"`	aggiunta in fondo (*append*)
`"r+"`	lettura e scrittura di un file esistente
`"rb"`, `"wb"`, `"ab"`…	come sopra, per file **binari**

File di testo: I/O formattato

Si usano come `printf` / `scanf`, ma con il file come primo argomento: `fprintf(f, ...)` e `fscanf(f, ...)`.

codice/file.c

```c
/* scrive n interi su un file di testo, uno per riga */
int scriviNumeri(char *nomeFile, int v[], int n) {
  FILE *f;
  int i;
  f = fopen(nomeFile, "w");        /* "w": crea o SVUOTA il file        */
  if (f == NULL)
    return 0;                      /* SEMPRE controllare l'apertura!    */
  for (i = 0; i < n; i++)
    fprintf(f, "%d\n", v[i]);
  fclose(f);
  return 1;
}
```

Per leggere "fino alla fine" si usa il **valore restituito da** `fscanf` (quanti dati ha letto): quando i dati finiscono restituisce `EOF` (un valore negativo) o comunque un numero diverso da quelli richiesti.

codice/file.c

```c
/* legge interi da un file di testo finche' ce ne sono; ne restituisce la somma */
int sommaDaFile(char *nomeFile, int *pquanti) {
  FILE *f;
  int x, somma = 0;
  *pquanti = 0;
  f = fopen(nomeFile, "r");
  if (f == NULL)
    return 0;
  while (fscanf(f, "%d", &x) == 1) { /* fscanf restituisce quanti dati ha letto */
    somma += x;
    (*pquanti)++;
  }
  fclose(f);
  return somma;
}
```

FEOF

Esiste anche `feof(f)`, che diventa vero **dopo** che una lettura ha provato a superare la fine. Il ciclo `while (!feof(f)) { fscanf(...); ... }` elabora l'ultimo dato **due volte** se il file finisce con un a capo. Il controllo sul valore di `fscanf` è più sicuro.

Altre funzioni utili: `fgetc(f)` / `fputc(c, f)` per un carattere alla volta; `fgets(buffer, dim, f)` per una riga intera.

Lo **standard input** e **output** sono anch'essi file già aperti: `stdin`, `stdout`. `printf(...)` equivale a `fprintf(stdout, ...)`.

File binari

Un file di testo contiene caratteri (il numero 1216 occupa 4 caratteri). Un file **binario** contiene i byte **così come sono in memoria** (un `int` occupa sempre `sizeof(int)` byte). È più compatto e veloce, ma non si legge con un editor di testo.

- `fwrite(indirizzo, dimensione_elemento, quanti, f)`: scrive `quanti` elementi presi dall'indirizzo dato; restituisce quanti ne ha scritti.
- `fread(indirizzo, dimensione_elemento, quanti, f)`: legge fino a `quanti` elementi; restituisce **quanti ne ha letti** (0 alla fine del file).
- `fseek(f, spostamento, origine)`: sposta la posizione corrente (accesso **diretto**, *random*). Origine: `SEEK_SET` (inizio), `SEEK_CUR` (posizione attuale), `SEEK_END` (fine).
- `ftell(f)`: posizione corrente in byte; `rewind(f)`: torna all'inizio.

codice/file.c

```c
/* file binari: si scrivono/leggono BLOCCHI di byte, cosi` come sono in memoria */
int salvaVoli(char *nomeFile, TipoVolo voli[], int n) {
  FILE *f = fopen(nomeFile, "wb");
  int scritti;
  if (f == NULL)
    return 0;
  scritti = fwrite(voli, sizeof(TipoVolo), n, f);  /* n elementi in un colpo */
  fclose(f);
  return scritti == n;
}

int caricaVoli(char *nomeFile, TipoVolo voli[], int max) {
  FILE *f = fopen(nomeFile, "rb");
  int letti;
  if (f == NULL)
    return 0;
  letti = fread(voli, sizeof(TipoVolo), max, f);   /* restituisce quanti ne ha letti */
  fclose(f);
  return letti;
}

/* accesso diretto: leggo SOLO il k-esimo volo (k da 0) */
int leggiVoloK(char *nomeFile, int k, TipoVolo *pv) {
  FILE *f = fopen(nomeFile, "rb");
  int ok;
  if (f == NULL)
    return 0;
  fseek(f, (long) k * sizeof(TipoVolo), SEEK_SET);  /* salto k record   */
  ok = fread(pv, sizeof(TipoVolo), 1, f) == 1;
  fclose(f);
  return ok;
}
```

Con un file binario di record tutti uguali si può leggere direttamente il k-esimo record, senza leggere quelli prima: è il vantaggio dell'accesso diretto.

ERRORI FREQUENTI CON I FILE

- Non controllare se `fopen` ha restituito `NULL`.
- Dimenticare `fclose` (i dati scritti possono restare nel buffer e andare persi).
- Aprire con `"w"` un file che si voleva solo leggere: lo si svuota.
- Salvare in un file binario una struct che contiene un puntatore (`char *descrizione`): si salva l'**indirizzo**, non la stringa! Al prossimo avvio quell'indirizzo non vale più nulla. Le stringhe esatte vanno salvate come testo, oppure scrivendo prima la lunghezza e poi i caratteri.

ESERCIZIO 7.1

Scrivi una funzione `int copiaPositivi(char *origine, char *destinazione)` che legge interi da un file di testo e scrive in un altro file solo i positivi, uno per riga. Restituisce quanti numeri ha copiato, oppure -1 se un file non si apre.

Soluzione

```c
int copiaPositivi(char *origine, char *destinazione) {
  FILE *fin, *fout;
  int x, copiati = 0;
  fin = fopen(origine, "r");
  if (fin == NULL)
    return -1;
  fout = fopen(destinazione, "w");
  if (fout == NULL) {
    fclose(fin);                    /* chiudo quello gia` aperto */
    return -1;
  }
  while (fscanf(fin, "%d", &x) == 1)
    if (x > 0) {
      fprintf(fout, "%d\n", x);
      copiati++;
    }
  fclose(fin);
  fclose(fout);
  return copiati;
}
```

CAPITOLO 8

Puntatori

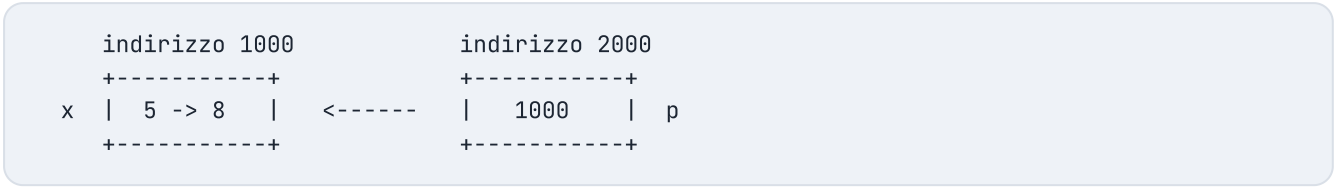
(slide lezioni 13 e 14, complementi lezione 13; testo1 cap. 14)

I puntatori sono **l'argomento chiave** del corso: senza di loro non si fanno né i parametri di output né le liste. Vale la pena studiarli con calma e con tanti disegni.

Indirizzi e puntatori

Ogni variabile occupa una locazione di memoria che ha un **indirizzo**. Un **puntatore** è una variabile che **contiene un indirizzo**.

```
int x = 5;
int *p;          /* p e` un "puntatore a int": puo` contenere l'indirizzo di un int */
p = &x;          /* &x = indirizzo di x: ora p "punta a" x                           */
*p = 8;          /* *p = "la variabile puntata da p", cioe` x: ora x vale 8          */
```



- `&` (**indirizzo di**): applicato a una variabile, dà il suo indirizzo.
- `*` (**dereferenziazione**): applicato a un puntatore, dà la variabile puntata.
- Nella **dichiarazione** `int *p;` l'asterisco dice solo che `p` è un puntatore.
- `NULL` (definito in `stdio.h` e `stdlib.h`) è il puntatore che **non punta a niente**. Dereferenziare `NULL` fa andare in crash il programma.

PUNTATORE NON INIZIALIZZATO

`int *p; *p = 3;` scrive in un indirizzo casuale. Un puntatore va **sempre** inizializzato: a `NULL`, all'indirizzo di una variabile, o al risultato di `malloc`.

Parametri di output (passaggio per indirizzo)

Per permettere a una funzione di **modificare una variabile del chiamante**, le si passa l'**indirizzo** della variabile. La funzione lavora con `*parametro`.

codice/basi.c

```c
void scambioNO(int a, int b) {     /* SBAGLIATA: scambia due COPIE      */
  int t = a;
  a = b;
  b = t;
}

void scambio(int *pa, int *pb) {   /* GIUSTA: lavora sugli originali    */
  int t = *pa;
  *pa = *pb;
  *pb = t;
}
```

```
chiamata scambio(&a, &b) con a = 3, b = 7:

  RDA scambio:  pa = &a    pb = &b    t = 3
                 |          |
                 v          v
  RDA main:     a: 3 -> 7   b: 7 -> 3
```

È lo stesso meccanismo di `scanf("%d", &x)`: `scanf` deve scrivere dentro `x`, quindi le serve il suo indirizzo.

Schema per gli esercizi d'esame ("la funzione restituisca A, B e C"): uno dei risultati con `return`, gli altri come parametri puntatore.

```c
/* restituisce quanti; in *media e *massimo gli altri due risultati */
int analizza(TipoLista l, int soglia, double *media, int *massimo);

/* nel chiamante: */
n = analizza(lista, 10, &m, &mx);
```

PARENTESI CON ++

Dentro la funzione, per incrementare un contatore di output scrivi `(*conta)++;`. Senza parentesi `*conta++` incrementa il **puntatore** (precedenza di `++`) e non il valore. In alternativa: `*conta = *conta + 1;`.

Puntatori e array

Il **nome di un array** è l'indirizzo del suo primo elemento: `v` equivale a `&v[0]`.

Aritmetica dei puntatori: se `p` punta a un elemento di un array, `p + 1` punta all'elemento **successivo** (il compilatore aggiunge `sizeof(tipo)` byte, non 1). Quindi:

Notazione con indice	Notazione con puntatori
`v[i]`	`*(v + i)`
`&v[i]`	`v + i`

```
int v[5] = {10, 20, 30, 40, 50};
int *p;
for (p = v; p < v + 5; p++)     /* scansione con un puntatore */
  printf("%d ", *p);
```

Differenze tra puntatori allo stesso array danno il numero di elementi tra loro (`p - v`). Ecco come lavora `strlen` :

codice/basi.c

```
/* come lavora strlen: conta i caratteri fino al terminatore '\0' */
int miaStrlen(char *s) {
  int n = 0;
  while (s[n] != '\0')
    n++;
  return n;
}

/* versione con aritmetica dei puntatori */
int miaStrlenPunt(char *s) {
  char *p = s;
  while (*p != '\0')
    p++;
  return p - s;                    /* differenza tra puntatori          */
}
```

Differenza importante: un puntatore è una variabile e si può spostare (`p++`); il nome di un array è un indirizzo costante (`v++` è un errore).

Funzioni che restituiscono puntatori

Una funzione può restituire un indirizzo, ad esempio quello di un elemento trovato in un array, o `NULL` se non lo trova:

```
int *cercaPrimoNegativo(int v[], int dim) {
  int i;
  for (i = 0; i < dim; i++)
    if (v[i] < 0)
      return &v[i];             /* indirizzo di un elemento dell'array del chiamante: ok */
  return NULL;
}
```

Va bene restituire l'indirizzo di qualcosa che **sopravvive** alla funzione (un elemento dell'array del chiamante, memoria allocata con `malloc`), **mai** quello di una variabile locale.

Puntatori a struct

Con le struct (capitolo 10) e le liste userai continuamente l'operatore **freccia**:

```
p->campo      /* equivale a  (*p).campo  : il campo della struct puntata da p */
```

Puntatori a puntatori

Se una funzione deve **modificare un puntatore** del chiamante (ad esempio la testa di una lista), le si passa l'indirizzo del puntatore: un **puntatore a puntatore**.

```
void azzera(int **pp) {
  *pp = NULL;                 /* modifica il puntatore del chiamante */
}

int *q = &x;
azzera(&q);                   /* ora q vale NULL */
```

Con le liste, `TipoLista *pl` è proprio questo: `TipoLista` è già un puntatore, `TipoLista *` è l'indirizzo di quel puntatore (capitolo 14).

LA REGOLA D'ORO DEI PARAMETRI

Se una funzione deve **cambiare** una variabile del chiamante, ricevi **l'indirizzo** di quella variabile. Una lista di cui cambia la **testa** (inserimento in testa, eliminazione del primo, inserimento ordinato) si passa come `TipoLista *`. Una lista che si **legge soltanto**, o di cui si modificano solo i nodi interni, si passa come `TipoLista`.

ESERCIZIO 8.1 (ESECUZIONE SIMULATA)

Cosa stampa?

```
int a = 1, b = 2, *p, *q;
p = &a;
q = &b;
*p = *q + 10;
p = q;
*p = *p * 3;
printf("%d %d %d %d\n", a, b, *p, *q);
```

Soluzione

`*p = *q + 10` mette in `a` il valore 2 + 10 = 12. Poi `p = q`: ora anche `p` punta a `b`. `*p = *p * 3` mette in `b` il valore 6. Stampa: `12 6 6 6`.

ESERCIZIO 8.2

Scrivi `void statistiche(int v[], int dim, int *somma, int *pari, double *media)` che calcola somma, numero di elementi pari e media dell'array.

Soluzione

```c
void statistiche(int v[], int dim, int *somma, int *pari, double *media) {
  int i;
  *somma = 0;
  *pari = 0;
  for (i = 0; i < dim; i++) {
    *somma += v[i];
    if (v[i] % 2 == 0)
      (*pari)++;
  }
  if (dim > 0)
    *media = (double) *somma / dim;
  else
    *media = 0;
}
/* chiamata:  statistiche(v, N, &s, &np, &m); */
```

CAPITOLO 9

Stringhe e allocazione dinamica della memoria

(slide lezioni 15 e 16, EG5, video "ArrayDouble"; testo1 cap. 13, 14, app. B)

Le stringhe in C

Una **stringa** è un **array di `char` terminato dal carattere `'\0'`** (codice 0). Il `'\0'` segna la fine: per questo una stringa di k caratteri occupa **k+1** caratteri.

```
char s[10] = "POCO";        /* 10 posti, ne usa 5: P O C O \0 */
```

```
indice:  0   1   2   3   4    5   6   7   8   9
       +---+---+---+---+----+---+---+---+---+---+
   s   | P | O | C | O | \0 | ? | ? | ? | ? | ? |
       +---+---+---+---+----+---+---+---+---+---+
```

Lettura e stampa: `scanf("%s", s);` (senza `&`, perché `s` è già un indirizzo; legge fino al primo spazio) e `printf("%s", s);`. Per leggere una riga con spazi: `fgets(s, 10, stdin);` (che però tiene anche il `'\n'` finale).

Le funzioni di `string.h`

Funzione	Cosa fa
`strlen(s)`	numero di caratteri **escluso** il `'\0'`
`strcpy(dest, src)`	copia `src` in `dest` (dest deve avere spazio!)
`strncpy(dest, src, n)`	copia al massimo n caratteri (può non mettere il `'\0'`)
`strcmp(a, b)`	**0** se uguali, **< 0** se a viene prima in ordine alfabetico, **> 0** se dopo
`strcat(dest, src)`	aggiunge `src` in fondo a `dest`

LE STRINGHE NON SI CONFRONTANO CON == E NON SI COPIANO CON =

`if (s1 == s2)` confronta gli **indirizzi**, non il contenuto: si usa `strcmp(s1, s2) == 0`. E `s1 = s2;` tra array non compila (oppure, tra puntatori, copia l'indirizzo e non i caratteri): si usa `strcpy`. Questo errore all'esame costa caro, perché i testi chiedono sempre di cercare per nome, codice o grado.

Allocazione statica e dinamica

- **Statica / automatica**: la dimensione è decisa **quando scrivi il programma** (`int v[100];`). Le variabili locali vivono nell'RDA e muoiono con la funzione.

- **Dinamica**: la memoria si chiede **durante l'esecuzione**, quando si sa quanta ne serve, in una zona chiamata **heap**. Rimane allocata finché non la liberi tu con `free`, **anche dopo la fine della funzione** che l'ha chiesta.

Funzioni di `stdlib.h`:

Funzione	Cosa fa
`malloc(nbyte)`	alloca `nbyte` byte e restituisce l'indirizzo (contenuto casuale), `NULL` se non c'è memoria
`calloc(n, dim)`	alloca n elementi di `dim` byte e li azzera
`realloc(p, nuovaDim)`	cambia la dimensione del blocco puntato da p (può spostarlo: usa l'indirizzo restituito)
`free(p)`	restituisce al sistema il blocco puntato da p

```
int *v;
int n;
scanf("%d", &n);
v = malloc(n * sizeof(int));     /* array dinamico di n interi          */
if (v == NULL) {                 /* SEMPRE controllare ("test sull'allocazione") */
  printf("memoria esaurita\n");
  return 0;
}
v[0] = 3;                        /* si usa come un array normale         */
...
free(v);                         /* quando non serve piu`                */
```

codice/basi.c

```
/* crea un array dinamico di n double, tutti inizializzati a val */
double *creaArray(int n, double val) {
  int i;
  double *a = malloc(n * sizeof(double));
  if (a == NULL)
    return NULL;
  for (i = 0; i < n; i++)
    a[i] = val;                    /* si usa come un array normale      */
  return a;                        /* la memoria SOPRAVVIVE alla funzione */
}
```

I TRE ERRORI DELLA MEMORIA DINAMICA

1. **Non controllare** `NULL` dopo `malloc`.
2. **Memory leak**: perdere l'ultimo puntatore a un blocco senza averlo liberato (la memoria resta occupata fino alla fine del programma).
3. **Usare memoria già liberata** (*dangling pointer*): dopo `free(p)`, `p` contiene ancora il vecchio indirizzo ma la memoria non è più tua. Se serve, scrivi `p = NULL;` subito dopo.

Stringhe "esatte"

Una **stringa esatta** è allocata con **esattamente** lo spazio necessario: `strlen + 1` caratteri. Nessuno spreco. È **la tecnica richiesta in tutti i testi d'esame** per i campi "stringa di lunghezza non nota a priori".

Duplicato esatto di una stringa

codice/basi.c

```c
/* restituisce una copia "esatta" di s (NULL se manca memoria) */
char *duplicato(char *s) {
  char *nuova = malloc(strlen(s) + 1);   /* +1 per il '\0'!          */
  if (nuova != NULL)
    strcpy(nuova, s);
  return nuova;
}
```

```
prima:    str ---> | P | O | C | O | \0 | ? | ? | ? | ? |   (array di 10)

malloc(strlen(s)+1) = malloc(5):
          nuova -> | ? | ? | ? | ? | ?  |

strcpy:   nuova -> | P | O | C | O | \0 |   5 caratteri, nessuno sprecato
```

La memoria allocata con `malloc` **sopravvive** alla fine di `duplicato`: l'RDA sparisce (con la variabile `nuova`), ma il blocco resta, e il suo indirizzo è stato restituito al chiamante.

Lo schema con il buffer

Per leggere una stringa di lunghezza ignota:

1. si legge in un **buffer** abbastanza grande (`char buffer[LUNGMAX+1]`);
2. si alloca con `malloc(strlen(buffer) + 1)`;
3. si copia con `strcpy`;
4. il buffer si riusa per la prossima lettura.

```c
#define LUNGMAX 80
char buffer[LUNGMAX + 1];
char *nome;

scanf("%s", buffer);
nome = malloc(strlen(buffer) + 1);
if (nome != NULL)
  strcpy(nome, buffer);
```

Array di stringhe

Un **array di stringhe** è un **array di puntatori a char**: ogni elemento punta a una stringa esatta allocata a parte.

```c
#define N 6
char *arrStr[N];           /* 6 puntatori; le stringhe vanno allocate */
```

```
arrStr
 [0] ---> | P | O | C | O | \0 |
 [1] ---> | O | R | O | \0 |
 [2] ---> | R | E | S | T | O | \0 |
 ...
 [5] ---> | P | R | O | M | O | Z | i | o | n | \0 |
```

Lettura di N stringhe esatte (dalle slide della lezione 16):

```c
void costruisciArrayStringhe(char *v[], int n) {
  char buffer[LUNGMAX + 1];
  int i;
  for (i = 0; i < n; i++) {
    printf("scrivi una stringa: ");
    scanf("%s", buffer);
    v[i] = malloc(strlen(buffer) + 1);
    if (v[i] != NULL)
      strcpy(v[i], buffer);
    else {
      printf("memoria esaurita\n");
      break;
    }
  }
}
```

Ricerca in un array di stringhe (con flag, fermandosi appena trovata):

```c
int presenteIn(char *cercata, char **v, int dim) {   /* char *v[] == char **v */
  int trovata = 0, i;
  for (i = 0; i < dim && trovata == 0; i++)
    if (strcmp(cercata, v[i]) == 0)
      trovata = 1;
  return trovata;
}
```

Nel **tipo** dei parametri, `char v[]` equivale a `char *v`, e quindi `char *v[]` equivale a `char **v`.

Array dinamici che crescono: realloc

Se un array dinamico è pieno, lo si può ingrandire (è la tecnica di `voli6.c` nelle EG):

```c
int *nuovo = realloc(v, 2 * dim * sizeof(int));  /* raddoppio */
if (nuovo != NULL) {
  v = nuovo;          /* uso il nuovo indirizzo (il blocco puo` essersi spostato) */
  dim = 2 * dim;
}                     /* se fallisce, v e` ancora valido e intatto */
```

ESERCIZIO 9.1

Scrivi `char *concatenaEsatta(char *a, char *b)` che restituisce una **nuova** stringa esatta contenente a seguita da b (le due stringhe originali non vanno toccate).

Soluzione

```c
char *concatenaEsatta(char *a, char *b) {
  char *r = malloc(strlen(a) + strlen(b) + 1);   /* un solo '\0' */
  if (r != NULL) {
    strcpy(r, a);
    strcat(r, b);
  }
  return r;
}
```

ESERCIZIO 9.2

Scrivi `int contaVocali(char *s)` senza usare `strlen` (scorri fino al `'\0'`).

Soluzione

```c
int contaVocali(char *s) {
  int i, n = 0;
  char c;
  for (i = 0; s[i] != '\0'; i++) {
    c = s[i];
    if (c >= 'A' && c <= 'Z')
      c = c + 32;                    /* in minuscolo */
    if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u')
      n++;
  }
  return n;
}
```

CAPITOLO 10

Struct, typedef e tabelle

(slide lezioni 18 e 19, EG6 e EG9; testo1 cap. 20; testo2 cap. 1-2)

Struct: il record

Un **array** raccoglie dati **dello stesso tipo**. Una **struct** (record) raccoglie dati di **tipi diversi** che descrivono **una cosa sola**. Ogni componente si chiama **campo**.

```
struct ora {
  int ore, minuti;
};

struct volo {
  char codice[6];              /* 5 caratteri + '\0': lunghezza fissa  */
  char *destinazione;          /* stringa esatta: lunghezza ignota     */
  struct ora oraPartenza;      /* struct dentro una struct             */
  int postiLiberi;
};
```

Accesso ai campi con il **punto**:

```
struct volo v;
strcpy(v.codice, "AZ123");
v.oraPartenza.ore = 10;
v.oraPartenza.minuti = 35;
v.postiLiberi = 120;
```

A differenza degli array, le struct **si possono assegnare** con `=` (si copiano tutti i campi) e **passare/restituire per valore**. Non si possono però confrontare con `==`.

COPIA DI UNA STRUCT CON UN PUNTATORE DENTRO

Se `a` e `b` sono `struct volo`, `b = a;` copia il **puntatore** `destinazione`, non la stringa: dopo l'assegnamento `a.destinazione` e `b.destinazione` puntano alla **stessa** stringa. Se poi fai `free` di una delle due, l'altra resta con un puntatore non valido.

typedef: dare un nome ai tipi

`typedef tipo_esistente NuovoNome;` crea un **sinonimo** di tipo. Il corso lo usa sempre, con nomi che iniziano per `Tipo`:

```c
typedef struct volo TipoVolo;

typedef struct {              /* struct "anonima" con nome dato dal typedef */
  double x, y;
  char colore;
} TipoPunto;
```

Struct e funzioni

codice/basi.c

```c
struct punto {
  double x, y;
  char colore;                    /* 'R', 'G', 'B' ...                */
};
typedef struct punto TipoPunto;

void stampaPunto(TipoPunto p) {    /* per valore: riceve una copia      */
  printf("(%.1f, %.1f) colore %c\n", p.x, p.y, p.colore);
}

void trasla(TipoPunto *pp, double dx, double dy) {  /* per indirizzo    */
  pp->x += dx;                    /* pp->x  equivale a  (*pp).x        */
  pp->y += dy;
}
```

- **Per valore** (`TipoPunto p`): la funzione riceve una **copia**. Va bene per leggere i dati.
- **Per indirizzo** (`TipoPunto *pp`): la funzione può **modificare** l'originale. Si accede ai campi con la **freccia**: `pp->x` equivale a `(*pp).x` (le parentesi servono perché il punto ha precedenza sull'asterisco).

Tabelle: array di struct

Una **tabella** è una struttura dati che rappresenta un **archivio** del mondo reale: una collezione di elementi dello stesso tipo (record). Le operazioni tipiche sono:

- **aggiunta** di un elemento;
- **eliminazione** di un elemento;
- **modifica** dei dati di un elemento;
- **ricerca** di un elemento tramite la sua **chiave**.

La **chiave di ricerca** è il campo che identifica un elemento in modo univoco (codice del volo, matricola, codice fiscale).

La tabella dei voli (EG9)

Realizzazione con array parzialmente riempito: un **sostegno** di `MAXVOLI` elementi e un contatore.

```c
#define MAXVOLI 10

typedef struct {
  TipoVolo arrayVoli[MAXVOLI];   /* sostegno di memoria                 */
  int quantiVoli;                /* quanti voli sono davvero in tabella  */
} TipoTabella;
```

I voli presenti sono sempre i primi `quantiVoli`; ogni scansione va da 0 a `quantiVoli - 1`.

Ricerca per chiave (restituisce l'indice, o -1):

```c
int indiceVolo(TipoTabella t, char cod[]) {
  int trovato = 0, i = 0;
  while (!trovato && i < t.quantiVoli)
    if (strcmp(t.arrayVoli[i].codice, cod) == 0)
      trovato = 1;
    else
      i++;
  if (trovato)
    return i;
  return -1;
}
```

Aggiunta (la tabella cambia: si passa per indirizzo):

```c
int aggiungiVolo(TipoTabella *t, TipoVolo v) {
  if (t->quantiVoli == MAXVOLI)          /* tabella piena                */
    return 0;
  if (indiceVolo(*t, v.codice) != -1)    /* chiave gia` presente         */
    return 0;
  t->arrayVoli[t->quantiVoli] = v;       /* nel primo posto libero       */
  t->quantiVoli++;
  return 1;
}
```

Eliminazione: si cerca l'indice k; si può spostare indietro di un posto tutti gli elementi dopo k, oppure (se l'ordine non conta) mettere l'**ultimo** al posto di k. Si libera la stringa esatta dell'eliminato.

```c
int eliminaVolo(TipoTabella *t, char cod[]) {
  int k = indiceVolo(*t, cod);
  if (k == -1)
    return 0;
  free(t->arrayVoli[k].destinazione);                    /* stringa esatta */
  t->arrayVoli[k] = t->arrayVoli[t->quantiVoli - 1];     /* l'ultimo al suo posto */
  t->quantiVoli--;
  return 1;
}
```

LIMITI DELLA TABELLA CON ARRAY (E PERCHÉ ARRIVANO LE LISTE)

Con l'array il numero massimo di elementi è fissato (`MAXVOLI`): o si spreca memoria o si rischia di riempirla. Con un **array dinamico** e `realloc` si può farla crescere (`voli6.c`), ma ogni ingrandimento copia tutto. Inserire o togliere in mezzo richiede di **spostare** molti elementi. La **lista collegata** (capitolo 14) risolve questi problemi: cresce un nodo alla volta e si inserisce o elimina **senza spostare nulla**. Per questo all'esame la "tabella" va sempre realizzata come **lista**.

Tipi di dato astratti (ADT)

Un **tipo di dato astratto** si definisce dicendo **che cosa** si può fare con i dati (le operazioni e il loro significato), **senza dire come** sono memorizzati. Esempio: l'ADT *Tabella* ha le operazioni aggiungi, elimina, cerca, modifica. Poi si sceglie una **rappresentazione concreta**: array di struct, array dinamico, lista collegata... Il resto del programma usa solo le funzioni, quindi si può cambiare la rappresentazione senza riscrivere tutto.

L'ADT **Lista** è una sequenza di elementi con operazioni come: crea lista vuota, è vuota?, inserisci in testa/in coda/in ordine, elimina, cerca, lunghezza. Si può rappresentare con un array (cenni a lezione) o con **struct e puntatori** (la rappresentazione che usiamo).

ALL'ORALE

"Che cos'è una tabella e quali operazioni ha?" "Che cos'è una chiave?" "Che differenza c'è tra tipo astratto e rappresentazione concreta?" "Quali sono i limiti di una tabella realizzata con array?"

ESERCIZIO 10.1

Usando `TipoTabella` dei voli, scrivi `int prenota(TipoTabella *t, char cod[], int k)` che prenota k posti sul volo indicato se ci sono abbastanza posti liberi. Restituisce 1 se riesce, 0 altrimenti.

Soluzione

```
int prenota(TipoTabella *t, char cod[], int k) {
  int i = indiceVolo(*t, cod);
  if (i == -1 || t->arrayVoli[i].postiLiberi < k)
    return 0;
  t->arrayVoli[i].postiLiberi -= k;
  return 1;
}
```

CAPITOLO 11

Ricorsione

(slide lezione 17, EG8, complementi lezione 8 sui record di attivazione; testo1 cap. 15)

L'idea

Una funzione è **ricorsiva** quando **chiama se stessa**. Funziona se:

1. c'è almeno un **caso base**, che si risolve **direttamente**, senza chiamate ricorsive;
2. il **caso ricorsivo** riduce il problema a uno **dello stesso tipo ma più piccolo**, e si avvicina sempre al caso base.

Il modo di pensare: "**supponiamo** che la funzione sappia già risolvere il problema più piccolo; come uso quel risultato per risolvere il problema grande?"

Il fattoriale

n! = n × (n-1) × ... × 1, e per definizione 0! = 1. Ma n! = n × (n-1)! : ecco la ricorsione.

codice/ricorsione_ordinamento.c

```c
int fatt(int n) {
  if (n <= 1)                    /* caso base                        */
    return 1;
  return n * fatt(n - 1);        /* caso ricorsivo: problema piu` piccolo */
}
```

Versione **iterativa**, per confronto:

```c
int fattIter(int n) {
  int i, prod = 1;
  for (i = 2; i <= n; i++)
    prod *= i;
  return prod;
}
```

Con `int` il fattoriale va in overflow già con n = 13 (13! supera 2^{31}-1); con `long` a 64 bit si arriva a 20.

Visualizzare la ricorsione con i record di attivazione

Ogni chiamata ricorsiva crea un **nuovo RDA**, con **i suoi** parametri e **le sue** variabili locali. Le RDA si impilano; quando si arriva al caso base, le chiamate terminano una alla volta **in ordine inverso**, ciascuna restituendo il valore alla precedente.

```
x = fatt(3)          FASE DI ANDATA (chiamate)          FASE DI RITORNO

                   +------------------+
                   | RDA fatt  n = 1  |  caso base ---> restituisce 1
                   +------------------+
                   | RDA fatt  n = 2  |  aspetta fatt(1) ---> 2 * 1 = 2
                   +------------------+
                   | RDA fatt  n = 3  |  aspetta fatt(2) ---> 3 * 2 = 6
                   +------------------+
                   | RDA main  x = ?  |  ---> x = 6
                   +------------------+
```

Visualizzazione logica (albero delle chiamate), altrettanto valida all'esame:

```
fatt(3) = 3 * fatt(2)
              fatt(2) = 2 * fatt(1)
                            fatt(1) = 1          (caso base)
              fatt(2) = 2 * 1 = 2
fatt(3) = 3 * 2 = 6
```

COME SI PRESENTA UN'ESECUZIONE SIMULATA CON RDA

1. Scrivi la chiamata iniziale e i valori dei parametri.
2. Disegna una RDA per ogni chiamata, impilandole, con **parametri**, **variabili locali** e **punto di ritorno** ("aspetta il risultato di ...").
3. Segna il **caso base** e il valore che restituisce.
4. Ripercorri la pila **all'indietro** scrivendo il valore restituito da ogni RDA e cancellando quelle terminate.
5. Scrivi il risultato finale.

Nelle tracce "Giovani Marmotte" (esercizio 3) è richiesto esattamente questo su una lista di 5 elementi: vedi il capitolo 17.

Inversione dell'input (EG8)

Leggere caratteri fino al punto e stamparli al contrario (`TOPI.` → `IPOT`). L'idea: leggo un carattere; **se non è il punto**, faccio leggere e invertire **il resto** e **poi** stampo il carattere letto.

codice/ricorsione_ordinamento.c

```c
/* legge caratteri fino al '.' e li stampa al contrario (es. TOPI. -> IPOT) */
void leggiEInverti(void) {
  char ch;
  scanf("%c", &ch);
  if (ch != '.') {                /* caso ricorsivo                    */
    leggiEInverti();              /* leggi e inverti il RESTO          */
    printf("%c", ch);             /* poi stampa questo carattere       */
  }                               /* caso base ('.'): non fare nulla   */
}
```

Ogni RDA conserva **il suo** `ch`: è la pila delle RDA a "ricordare" i caratteri, senza bisogno di un array.

```
input: T O P I .
RDA 1: ch='T'  chiama RDA 2 ... poi stampa T  (per ultimo)
RDA 2: ch='O'  chiama RDA 3 ... poi stampa O
RDA 3: ch='P'  chiama RDA 4 ... poi stampa P
RDA 4: ch='I'  chiama RDA 5 ... poi stampa I  (per primo)
RDA 5: ch='.'  caso base: non fa nulla e termina
output: I P O T
```

Palindromi (EG8)

Una parola è palindroma (radar, anna, ingegni) se si legge uguale nei due sensi. La porzione `p[i..j]` è palindroma se `p[i] == p[j]` **e** la porzione interna `p[i+1..j-1]` è palindroma. Caso base: `i >= j` (zero o un carattere: palindroma).

codice/ricorsione_ordinamento.c

```c
/* 1 se la porzione p[i..j] e` palindroma, 0 altrimenti */
int palin(char p[], int i, int j) {
  if (i >= j)                     /* niente (o un solo car.) da controllare */
    return 1;
  if (p[i] != p[j])
    return 0;
  return palin(p, i + 1, j - 1);   /* controllo la parte interna        */
}
```

Altri esempi

codice/ricorsione_ordinamento.c

```c
int mcdRic(int n, int m) {
  if (n == m)
    return n;
  if (m > n)
    return mcdRic(n, m - n);
  return mcdRic(n - m, m);
}
```

codice/ricorsione_ordinamento.c

```c
/* somma degli elementi v[0..n-1]: = v[n-1] + somma dei primi n-1 */
int sommaRic(int v[], int n) {
  if (n == 0)
    return 0;
  return v[n - 1] + sommaRic(v, n - 1);
}
```

Ricorsione sulle liste

Una lista è **vuota**, oppure è **un nodo seguito da una lista**. Per questo le funzioni ricorsive sulle liste sono naturali: caso base `l == NULL`, caso ricorsivo "faccio qualcosa col primo nodo e chiamo la funzione su `l->next`". Le trovi nel capitolo 14.

Ricorsione o iterazione?

- Tutto quello che si fa con la ricorsione si può fare con i cicli, e viceversa.
- La ricorsione è spesso **più semplice da scrivere e da capire** quando il problema è ricorsivo per natura (liste, alberi, divide et impera).
- Costa di più: ogni chiamata alloca un RDA (tempo e memoria). Con troppe chiamate annidate la pila si esaurisce (*stack overflow*).
- Esempio da evitare: Fibonacci "ingenuo" `fib(n) = fib(n-1) + fib(n-2)` ricalcola gli stessi valori un numero esponenziale di volte.

ERRORI TIPICI

- **Manca il caso base**, o il caso ricorsivo non si avvicina al caso base: ricorsione infinita, il programma va in crash.
- Dimenticare `return` davanti alla chiamata ricorsiva: `return 1 + conta(l->next);` e non solo `conta(l->next);`.
- Usare una variabile locale "per accumulare" tra le chiamate: ogni RDA ha la **sua** copia; il risultato si costruisce con i valori restituiti.

ESERCIZIO 11.1

Scrivi una funzione ricorsiva `int contaCifre(int n)` ($n \geq 0$) che restituisce il numero di cifre decimali di n, e mostra le RDA per `contaCifre(472)`.

Soluzione

```
int contaCifre(int n) {
  if (n < 10)                    /* caso base: una sola cifra */
    return 1;
  return 1 + contaCifre(n / 10); /* tolgo l'ultima cifra      */
}
```

```
RDA contaCifre n=4    caso base           -> 1
RDA contaCifre n=47   1 + contaCifre(4)   -> 1 + 1 = 2
RDA contaCifre n=472  1 + contaCifre(47)  -> 1 + 2 = 3
RDA main                                  -> riceve 3
```

ESERCIZIO 11.2

Scrivi una funzione ricorsiva `int maxArray(int v[], int n)` che restituisce il massimo dei primi n elementi (n ≥ 1).

Soluzione

```
int maxArray(int v[], int n) {
  int m;
  if (n == 1)
    return v[0];                /* un solo elemento: e` lui il massimo */
  m = maxArray(v, n - 1);       /* massimo dei primi n-1               */
  if (v[n - 1] > m)
    return v[n - 1];
  return m;
}
```

CAPITOLO 12

Ricerca e ordinamento

(slide lezione 20 e filmati sugli algoritmi di ordinamento; testo1 cap. 11, 12, caso di studio I; testo2 cap. 1-2)

Ricerca lineare (sequenziale)

Si scorre l'array dall'inizio finché si trova l'elemento o si arriva in fondo. Funziona **sempre**, anche se l'array non è ordinato. Nel caso peggiore fa **n** confronti.

codice/basi.c

```c
/* un array passato a una funzione e` l'indirizzo del suo primo elemento:
   la dimensione va passata a parte                                      */
int massimo(int v[], int dim) {
  int i, max = v[0];
  for (i = 1; i < dim; i++)
    if (v[i] > max)
      max = v[i];
  return max;
}

double media(int v[], int dim) {
  int i, somma = 0;
  for (i = 0; i < dim; i++)
    somma += v[i];
  return (double) somma / dim;     /* cast: altrimenti divisione intera! */
}

/* ricerca lineare con variabile "flag" (stile del corso):
   restituisce l'indice di x oppure -1                                   */
int indiceDi(int v[], int dim, int x) {
  int trovato = 0, i = 0;
  while (!trovato && i < dim)
    if (v[i] == x)
      trovato = 1;
    else
      i++;
  if (trovato)
    return i;
  return -1;
}
```

Ricerca binaria (dicotomica)

Funziona **solo su array ordinati**. Si guarda l'elemento **centrale**: se è quello cercato, fatto; se il cercato è più piccolo si continua nella metà sinistra, altrimenti nella destra. A ogni passo **si dimezza** lo spazio di ricerca: al massimo circa $\log_2 n$ confronti (per n = 1000, circa 10 invece di 1000).

codice/ricorsione_ordinamento.c

```c
/* l'array DEVE essere ordinato in modo crescente; restituisce indice o -1 */
int ricercaBinaria(int v[], int dim, int x) {
  int inizio = 0, fine = dim - 1, medio;
  while (inizio <= fine) {
    medio = (inizio + fine) / 2;
    if (v[medio] == x)
      return medio;
    if (x < v[medio])
      fine = medio - 1;            /* cerco nella meta` sinistra        */
    else
      inizio = medio + 1;          /* cerco nella meta` destra          */
  }
  return -1;
}
```

```
cerco 37 in  [10 13 14 29 37]
   inizio=0 fine=4  medio=2  v[2]=14 < 37  -> inizio=3
   inizio=3 fine=4  medio=3  v[3]=29 < 37  -> inizio=4
   inizio=4 fine=4  medio=4  v[4]=37       -> trovato, indice 4
```

Algoritmi di ordinamento

Ordinare = disporre gli elementi in ordine (di solito crescente). Esempio usato qui sotto: `29 10 14 37 13`.

Selection sort (ordinamento per selezione)

A ogni passo **seleziono il minimo** della parte non ancora ordinata e lo **scambio** con il primo elemento di quella parte.

codice/ricorsione_ordinamento.c

```c
void selectionSort(int v[], int dim) {
  int i, j, iMin, t;
  for (i = 0; i < dim - 1; i++) {
    iMin = i;                      /* cerco il minimo in v[i..dim-1]    */
    for (j = i + 1; j < dim; j++)
      if (v[j] < v[iMin])
        iMin = j;
    t = v[i]; v[i] = v[iMin]; v[iMin] = t;   /* lo porto in posizione i */
  }
}
```

```
29 10 14 37 13    minimo di tutto = 10, scambio con 29
10|29 14 37 13    minimo del resto = 13, scambio con 29
10 13|14 37 29    minimo = 14, gia` al suo posto
10 13 14|37 29    minimo = 29, scambio con 37
10 13 14 29 37    fine
```

Bubble sort (ordinamento a bolle)

Scorro l'array confrontando **coppie di elementi vicini** e le scambio se sono in ordine sbagliato. Dopo ogni passata il più grande "sale come una bolla" in fondo. Mi fermo quando una passata non fa scambi.

codice/ricorsione_ordinamento.c

```c
void bubbleSort(int v[], int dim) {
  int i, t, scambiato = 1, limite = dim - 1;
  while (scambiato) {              /* ripeto finche' faccio scambi      */
    scambiato = 0;
    for (i = 0; i < limite; i++)
      if (v[i] > v[i + 1]) {       /* coppia in ordine sbagliato        */
        t = v[i]; v[i] = v[i + 1]; v[i + 1] = t;
        scambiato = 1;
      }
    limite--;                      /* l'ultimo e` ormai al suo posto    */
  }
}
```

```
passata 1:  29 10 14 37 13 -> 10 29 14 37 13 -> 10 14 29 37 13 -> 10 14 29 37 13 -> 10 14 29
13 37
passata 2:  10 14 29 13|37 -> ... -> 10 14 13 29|37
passata 3:  10 14 13|29 37 -> 10 13 14|29 37
passata 4:  nessuno scambio: fine
```

Insertion sort (ordinamento per inserimento)

Come quando si ordinano le carte in mano: prendo un elemento alla volta e lo **inserisco al posto giusto** nella parte già ordinata a sinistra, spostando a destra quelli più grandi.

codice/ricorsione_ordinamento.c

```c
void insertionSort(int v[], int dim) {
  int i, j, x;
  for (i = 1; i < dim; i++) {
    x = v[i];                      /* elemento da inserire              */
    j = i - 1;
    while (j >= 0 && v[j] > x) {   /* sposto a destra i maggiori di x   */
      v[j + 1] = v[j];
      j--;
    }
    v[j + 1] = x;                  /* inserisco x nel buco              */
  }
}
```

```
29|10 14 37 13    inserisco 10: 10 29
10 29|14 37 13    inserisco 14: 10 14 29
10 14 29|37 13    inserisco 37: resta li`
10 14 29 37|13    inserisco 13: 10 13 14 29 37
```

Merge sort (cenni)

Divide et impera, ricorsivo: divido l'array a metà, ordino ricorsivamente le due metà, poi le **fondo** (*merge*) in un'unica sequenza ordinata, confrontando ogni volta i primi elementi delle due metà.

codice/ricorsione_ordinamento.c

```c
/* merge sort (ricorsivo): divido a meta`, ordino le due meta`, le fondo */
void fondi(int v[], int inizio, int medio, int fine) {
  int aux[100];                   /* per semplicita`: dim <= 100       */
  int i = inizio, j = medio + 1, k = 0;
  while (i <= medio && j <= fine)
    if (v[i] <= v[j]) aux[k++] = v[i++];
    else              aux[k++] = v[j++];
  while (i <= medio) aux[k++] = v[i++];
  while (j <= fine)  aux[k++] = v[j++];
  for (k = 0; k <= fine - inizio; k++)
    v[inizio + k] = aux[k];
}

void mergeSort(int v[], int inizio, int fine) {
  int medio;
  if (inizio < fine) {            /* almeno 2 elementi                 */
    medio = (inizio + fine) / 2;
    mergeSort(v, inizio, medio);
    mergeSort(v, medio + 1, fine);
    fondi(v, inizio, medio, fine);
  }
}
```

Quanto costano (complessità, cenni)

L'**efficienza** di un algoritmo si misura con il numero di operazioni in funzione della dimensione n dei dati (nel caso peggiore).

Algoritmo	Caso peggiore	Note
Ricerca lineare	proporzionale a n	anche su dati non ordinati
Ricerca binaria	proporzionale a log n	solo su dati ordinati
Selection sort	proporzionale a n^2	fa sempre gli stessi confronti
Bubble sort	proporzionale a n^2	veloce se l'array è quasi ordinato
Insertion sort	proporzionale a n^2	veloce se l'array è quasi ordinato
Merge sort	proporzionale a n log n	serve memoria aggiuntiva

I FILMATI DELLA LEZIONE 20

A lezione sono stati proposti quattro filmati (SORTING-ALG_A ... D) in cui persone "eseguono" gli algoritmi: guardali e prova a riconoscere l'algoritmo, poi controlla con le soluzioni sul sito. Un indizio: nel selection si cerca sempre il minimo e lo si scambia; nel bubble si confrontano solo vicini; nell'insertion si fa "scorrere" un elemento a sinistra.

ESERCIZIO 12.1

Applica a mano selection sort, bubble sort e insertion sort all'array `5 1 4 2 8`, scrivendo l'array dopo ogni passo.

Soluzione

Selection: `1|5 4 2 8` → `1 2|4 5 8` → `1 2 4|5 8` → `1 2 4 5|8`.

Bubble: passata 1: `1 5 4 2 8` → `1 4 5 2 8` → `1 4 2 5 8` → (5 e 8 ok) `1 4 2 5 8`; passata 2: `1 2 4 5 8`; passata 3: nessuno scambio, fine.

Insertion: inserisco 1 → `1 5|4 2 8`; inserisco 4 → `1 4 5|2 8`; inserisco 2 → `1 2 4 5|8`; inserisco 8 → `1 2 4 5 8`.

ESERCIZIO 12.2

Adatta il selection sort per ordinare la tabella dei voli (capitolo 10) per **codice** crescente.

Soluzione

```
void ordinaPerCodice(TipoTabella *t) {
  int i, j, iMin;
  TipoVolo tmp;
  for (i = 0; i < t->quantiVoli - 1; i++) {
    iMin = i;
    for (j = i + 1; j < t->quantiVoli; j++)
      if (strcmp(t->arrayVoli[j].codice, t->arrayVoli[iMin].codice) < 0)
        iMin = j;
    tmp = t->arrayVoli[i];                /* le struct si assegnano con = */
    t->arrayVoli[i] = t->arrayVoli[iMin];
    t->arrayVoli[iMin] = tmp;
  }
}
```

CAPITOLO 13

Qualità dei programmi e test di correttezza

(lezione 12, solo online, con video; slide "Qualità e Testing"; testo2 cap. 1-2)

Questo argomento esce spesso come **esercizio 3** ("descrivere il metodo a scatola nera / trasparente e applicarlo a `eser1`"). Vale fino a 7 punti e si prepara bene in anticipo.

Le qualità di un programma

- **Efficienza**: quanto **tempo** impiega e quanto **spazio** (memoria) usa. Si gestisce studiando l'algoritmo e la sua complessità.
- **Robustezza**: il programma funziona anche con **usi anomali** (input sbagliato, file mancante, memoria esaurita). Si gestisce controllando i dati in ingresso, chiedendo l'input in modo chiaro, controllando `NULL` dopo `malloc` e `fopen`.
- **Correttezza**: il programma **fa quello che ci si aspetta**. Si verifica con i **test**.

Gli errori

Tipo	Quando si scopre	Esempio
lessicale	compilazione	`returna x;` (parola che non esiste)
sintattico	compilazione	`foor (i=0; ...)`, `;` mancante
semantico	compilazione	`int i = 8.4;` (tipi incompatibili), variabile non dichiarata
logico / di codifica	**solo con i test**	`prod = prod + i` invece di `prod = prod * i`; `i <= i` invece di `i <= n`

Il compilatore segnala un errore **dopo** averlo trovato: la riga sbagliata può essere **prima** di quella indicata.

Un programma che compila **non è detto che sia corretto**. Bisogna testarlo.

Che cos'è un test

Test = (1) eseguire il programma con certi **dati di prova**; (2) confrontare l'**output atteso** (calcolato a mano prima) con l'**output effettivo**. Se coincidono si passa al test successivo; se no si fa **debugging**: si ispeziona il codice, si aggiungono stampe delle variabili (**metodo delle stampe**), si usa il **debugger** (watch sulle variabili, esecuzione passo passo, call stack).

Test fatti a caso non bastano. L'esempio del professore:

```c
int uguali(char p1[4], char p2[4]) {     /* SBAGLIATA */
  int u, i;
  for (i = 0; i < 4; i++)
    if (p1[i] != p2[i])
      u = 0;
    else
      u = 1;                             /* questo else e` l'errore */
  return u;
}
```

Test	Input	Atteso	Effettivo
1 (diverse)	PERA, TRAM	0	0
2 (uguali)	PERA, PERA	1	1
3 (diverse, ultima lettera uguale)	PERA, MELA	0	**1**

I primi due test passano, eppure la funzione è sbagliata: conta solo l'**ultimo** confronto. Serve un **metodo** per scegliere i test **prima** di farli. I metodi sono due.

Metodo a scatola nera (copertura dell'input)

Si scelgono i test **guardando solo la specifica** (che cosa deve fare la funzione), **senza guardare il codice**.

1. Si **partiziona** l'insieme dei possibili input in **classi di equivalenza**: input di una stessa classe dovrebbero essere trattati "allo stesso modo" (stesso tipo di output atteso).
2. Si sceglie **un rappresentante per classe**: un test per ogni classe.
3. Più le classi sono **specifiche**, più test si fanno e più si è sicuri. Bisogna considerare in particolare i **casi limite**: struttura vuota, un solo elemento, elemento cercato in prima / ultima posizione, assente, valori uguali alla soglia...

Esempio (lezione) per `indiceVolo(TipoTabella t, char *codice)`: tabella vuota; tabella piena e codice presente; tabella piena e codice assente; codice presente in prima posizione; codice presente in ultima posizione; codice presente in mezzo...

Metodo a scatola trasparente (copertura del codice)

Si scelgono i test **guardando il codice**, in modo che, nel complesso, i test **mettano in azione le sue parti**. Ci sono criteri via via più scrupolosi:

1. **Copertura delle istruzioni**: alla fine dei test **ogni istruzione** è stata eseguita almeno una volta.
2. **Copertura delle decisioni**: in più, ogni **decisione** (condizione di `if`, `while`, `for`) è stata **vera** almeno una volta **e falsa** almeno una volta.
3. **Copertura delle condizioni**: in più, ogni **condizione elementare** dentro una decisione composta (es. `A<10` e `C<20` in `A<10 || C<20`) è stata vera e falsa almeno una volta.

4. **Valori limite**: si aggiungono test con i valori **al limite** delle condizioni (es. 0, 10, 20) e **vicini** al limite (9, 11, 19, 21, -1, +1).

Per i cicli, si prova anche: **zero iterazioni**, **una** iterazione, **più** iterazioni.

L'esempio della lezione: si voleva "se A ≠ 0 stampa 1; se A < 10 **e** C < 20 stampa 2", ma si è scritto `if (A > 0)` e `if ((A < 10) || (C < 20))`. Il test A=3, C=18 copre tutte le istruzioni ma non trova gli errori; aggiungendo criteri e valori limite (A=-3, A=0, C=20, A=20...) gli errori emergono. **Nessun criterio garantisce di trovare tutti gli errori**: più criteri = più test = più costo, ma più sicurezza.

Test top-down (per programmi con molte funzioni)

Si fa un **test incrementale**: si prova una funzione alla volta, partendo dal `main`. Le funzioni non ancora provate si sostituiscono con **stub**: versioni finte che restituiscono direttamente il risultato giusto per il test in corso. Se qualcosa va storto, l'errore può essere solo nella funzione che si sta provando.

Come rispondere all'esercizio 3 (modello)

SCHEMA DI RISPOSTA

1. **Descrizione del metodo** (5-8 righe): obiettivo, idea (specifica vs codice), criteri (classi di equivalenza / i 4 criteri di copertura), limiti (nessun metodo garantisce la correttezza).
2. **Tabella dei test** applicata a `eser1` : per ogni test **perché** lo faccio (quale classe / quale istruzione o decisione copre), **dati di prova** concreti (una lista disegnata), **output atteso**.
3. **Esecuzione simulata** di almeno un test, mostrando come cambiano le variabili, e conclusione "output effettivo = atteso".

Una risposta completa, applicata alla funzione `eser1` della traccia dei guasti, è nel capitolo 17.

ESERCIZIO 13.1

Data la funzione `int maxArray(int v[], int dim)` (restituisce il massimo, dim ≥ 1), progetta i test a scatola nera.

Soluzione

Classi di equivalenza (partizione dell'input) e un rappresentante per ciascuna:

Test	Classe	Input	Atteso
1	un solo elemento	`{7}`	7
2	massimo in prima posizione	`{9, 2, 5}`	9
3	massimo in ultima posizione	`{2, 5, 9}`	9
4	massimo in mezzo	`{2, 9, 5}`	9
5	tutti negativi	`{-4, -2, -8}`	-2
6	massimo ripetuto	`{9, 1, 9}`	9
7	tutti uguali	`{3, 3, 3}`	3

Il test 5 scopre l'errore classico `max = 0` come valore iniziale; i test 2 e 3 scoprono errori sugli estremi del ciclo.

CAPITOLO 14

Liste collegate

(slide lezioni 21-25, EG10 e EG11; testo2 cap. 6)

IL CAPITOLO PIÙ IMPORTANTE

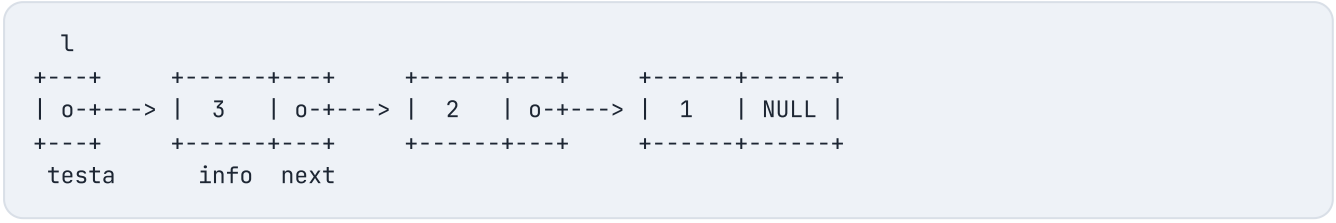
Gli esercizi 1 e 2 dello scritto (23 punti su 30) sono **sempre** sulle liste. Studia questo capitolo finché sai scrivere **a memoria e senza errori** tutte le funzioni qui sotto, disegnando i puntatori.

L'idea

Una **lista collegata** (o concatenata) è una sequenza di **nodi**. Ogni nodo è una struct allocata dinamicamente con:

- l'**informazione** (`info`): un intero, o una struct con tutti i dati dell'elemento;
- un **puntatore al nodo successivo** (`next`). L'ultimo nodo ha `next == NULL` .

La lista si identifica con il **puntatore al primo nodo** (la **testa**). La **lista vuota** è il puntatore `NULL` .



Rispetto all'array: la lista **cresce e si accorcia un nodo alla volta** (nessuna dimensione massima, nessuno spreco) e si inserisce/elimina **senza spostare** gli altri elementi; però per raggiungere il k-esimo elemento bisogna **scorrere** dall'inizio (niente accesso diretto `v[k]`).

I tipi

codice/liste.c

```c
typedef int TipoInfo;              /* il tipo dell'informazione nel nodo */

struct nodo {
  TipoInfo info;                   /* il dato                            */
  struct nodo *next;               /* puntatore al nodo successivo       */
};

typedef struct nodo TipoNodo;      /* un nodo (record)                   */
typedef TipoNodo *TipoLista;       /* una lista = puntatore al 1o nodo   */
```

- `struct nodo` contiene un puntatore a `struct nodo` : è una **struttura ricorsiva** (per questo serve il nome `struct nodo` e non basta il typedef, che non è ancora definito a quel punto).
- `TipoLista` è **un puntatore**. Una variabile lista si dichiara e **si inizializza sempre a** `NULL` : `TipoLista l = NULL;` .

NOMI E CONVENZIONI

Il libro Calvanese et al. usa nomi come `TipoSCL` (Sequenza Collegata Lineare) e `TipoNodoSCL`; a lezione si usano anche nomi come `insTestaLista`. I nomi non contano: conta la struttura. All'esame, se il testo chiede un nome (es. `TipoReggimento`), **usa quello**.

Quando passare `TipoLista` e quando `TipoLista *`

La funzione...	Parametro	Esempi
legge soltanto la lista	`TipoLista l`	stampa, lunghezza, ricerca, conteggi, massimo
può **cambiare la testa**	`TipoLista *pl` (si chiama con `&l`)	inserimento in testa/coda/ordinato, eliminazione, deallocazione

Dentro la funzione con `TipoLista *pl`, la testa è `*pl`. Per passarla a una funzione che legge soltanto: `stampaLista(*pl)`.

Scansione: stampa e lunghezza

Lo schema base: un puntatore **p** parte dalla testa e avanza con `p = p->next` finché diventa `NULL`.

codice/liste.c

```c
void stampaLista(TipoLista l) {
  TipoNodo *p = l;           /* p "scorre" la lista, l resta fermo */
  while (p != NULL) {
    printf("%d -> ", p->info);
    p = p->next;             /* avanzamento al nodo successivo     */
  }
  printf("NULL\n");
}
```

codice/liste.c

```c
int lunghezza(TipoLista l) {
  int n = 0;
  while (l != NULL) {        /* l e` una COPIA: possiamo muoverla  */
    n++;
    l = l->next;
  }
  return n;
}
```

`l` è un parametro passato per valore: spostarlo non modifica la lista del chiamante.

Ricerca

codice/liste.c

```c
/* restituisce il puntatore al primo nodo che contiene x, NULL se non c'e` */
TipoNodo *cerca(TipoLista l, TipoInfo x) {
  while (l != NULL && l->info != x)   /* l'ordine delle condizioni conta! */
    l = l->next;
  return l;
}
```

L'ordine `l != NULL && l->info != x` è fondamentale: se `l` è `NULL` la seconda condizione non viene valutata (cortocircuito).

Variante con massimo e parametro di output (restituisce 0 se la lista è vuota):

codice/liste.c

```c
/* parametro di output: il massimo va in *pmax; il valore restituito
   dice se il massimo esiste (lista non vuota)                        */
int maxLista(TipoLista l, int *pmax) {
  if (l == NULL)
    return 0;
  *pmax = l->info;
  for (l = l->next; l != NULL; l = l->next)
    if (l->info > *pmax)
      *pmax = l->info;
  return 1;
}
```

Inserimento in testa

codice/liste.c

```c
/* inserimento in testa: la testa cambia, quindi serve TipoLista *
   restituisce 1 se l'inserimento riesce, 0 se la malloc fallisce      */
int insTesta(TipoLista *pl, TipoInfo x) {
  TipoNodo *nuovo = malloc(sizeof(TipoNodo));   /* 1. alloco          */
  if (nuovo == NULL)
    return 0;
  nuovo->info = x;                              /* 2. riempio         */
  nuovo->next = *pl;                            /* 3. aggancio        */
  *pl = nuovo;                                  /* 4. nuova testa     */
  return 1;
}
```

```
PRIMA:   *pl --> [2] --> [1] --> NULL

1. nuovo = malloc(...)            nuovo --> [ ? | ? ]
2. nuovo->info = 3                nuovo --> [ 3 | ? ]
3. nuovo->next = *pl              nuovo --> [ 3 | o-]--> [2] --> [1] --> NULL
4. *pl = nuovo                    *pl -----> [ 3 ] --> [2] --> [1] --> NULL
```

L'ordine dei passi 3 e 4 è importante: se facessi prima `*pl = nuovo`, perderei l'indirizzo del vecchio primo nodo.

Costruire una lista di n elementi con inserimenti in testa: gli elementi risultano in **ordine inverso** rispetto all'inserimento.

```c
TipoLista l = NULL;
for (i = 1; i <= 3; i++)
  insTesta(&l, i);         /* l: 3 -> 2 -> 1 -> NULL */
```

Inserimento in coda

Bisogna arrivare all'**ultimo** nodo (quello con `next == NULL`) e attaccargli il nuovo. Caso speciale: se la lista è vuota, il nuovo nodo diventa la testa.

codice/liste.c

```c
/* inserimento in coda, versione "con caso speciale" per la lista vuota */
int insCoda(TipoLista *pl, TipoInfo x) {
  TipoNodo *nuovo, *p;
  nuovo = malloc(sizeof(TipoNodo));
  if (nuovo == NULL)
    return 0;
  nuovo->info = x;
  nuovo->next = NULL;              /* sara` l'ultimo                    */
  if (*pl == NULL)                 /* caso speciale: lista vuota        */
    *pl = nuovo;
  else {
    p = *pl;
    while (p->next != NULL)        /* mi fermo sull'ULTIMO nodo         */
      p = p->next;
    p->next = nuovo;
  }
  return 1;
}
```

Attenzione alla condizione `while (p->next != NULL)`: ci si ferma **sull'ultimo** nodo, non dopo (con `while (p != NULL)` si uscirebbe con `p == NULL` e non si saprebbe più a chi attaccare il nuovo nodo).

La tecnica del record generatore (RG)

Il caso speciale "lista vuota / primo nodo" esiste perché il primo nodo non ha un predecessore. Il **record generatore** è un nodo **fittizio** messo davanti alla lista: così **ogni** nodo vero ha un predecessore e il caso speciale

sparisce. Alla fine la testa vera è `rg.next`.

codice/liste.c

```c
/* inserimento in coda con la tecnica del RECORD GENERATORE (RG):
   un nodo fittizio messo in testa elimina il caso speciale            */
int insCodaRG(TipoLista *pl, TipoInfo x) {
  TipoNodo rg;                    /* record generatore (variabile locale) */
  TipoNodo *p, *nuovo;
  nuovo = malloc(sizeof(TipoNodo));
  if (nuovo == NULL)
    return 0;
  nuovo->info = x;
  nuovo->next = NULL;
  rg.next = *pl;                  /* RG davanti al primo nodo vero     */
  p = &rg;                        /* p parte dal RG: non e` mai NULL   */
  while (p->next != NULL)
    p = p->next;
  p->next = nuovo;                /* se la lista era vuota, p == &rg   */
  *pl = rg.next;                  /* la testa vera e` dopo il RG       */
  return 1;
}
```

```
rg (finto)       nodi veri
+---+---+       +---+---+     +---+------+
| ? | o-+-----> | 3 | o-+---> | 2 | NULL |
+---+---+       +---+---+     +---+------+
  ^ p parte da qui, quindi p non e` mai NULL
```

Il RG è comodo in **tutte** le operazioni in cui serve il predecessore: inserimento in coda, inserimento ordinato, eliminazione (anche di più nodi, vedi capitolo 18).

Inserimento ordinato

Lista ordinata in modo crescente: bisogna trovare il **primo nodo con info ≥ x** e inserire il nuovo nodo **prima** di lui. Per inserire "prima" serve il nodo **precedente**: si scorre con **due puntatori**, `prec` e `corr`.

codice/liste.c

```c
/* inserimento che mantiene la lista ordinata in modo CRESCENTE */
int insOrdinato(TipoLista *pl, TipoInfo x) {
  TipoNodo *nuovo, *prec, *corr;
  nuovo = malloc(sizeof(TipoNodo));
  if (nuovo == NULL)
    return 0;
  nuovo->info = x;
  prec = NULL;                    /* nodo che precede corr             */
  corr = *pl;
  while (corr != NULL && corr->info < x) {  /* cerco il punto giusto    */
    prec = corr;
    corr = corr->next;
  }
  nuovo->next = corr;             /* il nuovo nodo precede corr        */
  if (prec == NULL)               /* va in testa (lista vuota o x min) */
    *pl = nuovo;
  else
    prec->next = nuovo;
  return 1;
}
```

```
inserisco 6 in   *pl --> [1] --> [5] --> [7] --> [9] --> NULL

scansione:  prec=NULL corr=[1]   1 < 6, avanzo
            prec=[1]  corr=[5]   5 < 6, avanzo
            prec=[5]  corr=[7]   7 >= 6, mi fermo

aggancio:   nuovo->next = corr  ([6] -> [7])
            prec->next  = nuovo ([5] -> [6])

DOPO:       *pl --> [1] --> [5] --> [6] --> [7] --> [9] --> NULL
```

I casi da controllare: lista vuota (`prec` resta `NULL`: nuova testa), x più piccolo di tutti (`prec == NULL`: nuova testa), x più grande di tutti (`corr == NULL`: in coda), x in mezzo.

Per le stringhe la condizione diventa `strcmp(corr->info.nome, nome) < 0`.

Eliminazione di un elemento

Stessa tecnica dei due puntatori: si cerca il nodo da eliminare **ricordando il precedente**, poi lo si "scavalca" e si libera la memoria.

codice/liste.c

```c
/* elimina il PRIMO nodo che contiene x; restituisce 1 se eliminato, 0 se assente */
int elimina(TipoLista *pl, TipoInfo x) {
  TipoNodo *prec = NULL, *corr = *pl;
  while (corr != NULL && corr->info != x) {
    prec = corr;
    corr = corr->next;
  }
  if (corr == NULL)               /* x non trovato                    */
    return 0;
  if (prec == NULL)               /* era il primo nodo: cambia la testa */
    *pl = corr->next;
  else
    prec->next = corr->next;      /* "scavalco" corr                  */
  free(corr);                     /* restituisco la memoria           */
  return 1;
}
```

```
elimino 5 da   *pl --> [1] --> [5] --> [7] --> NULL

prec=[1] corr=[5]          prec->next = corr->next      free(corr)

               *pl --> [1] ----------> [7] --> NULL
                            [5] (liberato)
```

ERRORI TIPICI NELLE ELIMINAZIONI

- `free(corr)` **prima** di aver collegato `prec->next = corr->next`: dopo la `free` non si può più leggere `corr->next`.
- Dimenticare il caso "elimino il primo nodo": va cambiata la testa (`*pl = corr->next`), quindi il parametro deve essere `TipoLista *`.
- Se il nodo contiene stringhe esatte, liberarle **prima** del nodo: `free(corr->info.nome);` `free(corr);`. Altrimenti è un memory leak.

Deallocazione di tutta la lista

codice/liste.c

```c
void dealloca(TipoLista *pl) {
  TipoNodo *daLiberare;
  while (*pl != NULL) {
    daLiberare = *pl;
    *pl = (*pl)->next;            /* prima avanzo ...                 */
    free(daLiberare);             /* ... poi libero (mai il contrario!) */
  }
}
```

Bisogna salvare il nodo da liberare, **avanzare**, e poi liberare: dopo `free` non si può più seguire il suo `next`.

Funzioni ricorsive sulle liste

Una lista è **vuota** oppure è **un nodo seguito da una lista**. Le funzioni ricorsive seguono tutte lo stesso schema: caso base `l == NULL`, caso ricorsivo che usa `l->info` e chiama la funzione su `l->next`.

codice/liste.c

```c
/* una lista e` VUOTA oppure e` un NODO seguito da una LISTA:
   e` una struttura intrinsecamente ricorsiva                          */
int lunghezzaRic(TipoLista l) {
  if (l == NULL)                      /* caso base: lista vuota       */
    return 0;
  return 1 + lunghezzaRic(l->next);   /* 1 (questo nodo) + resto      */
}

int contaRic(TipoLista l, TipoInfo x) {
  if (l == NULL)
    return 0;
  if (l->info == x)
    return 1 + contaRic(l->next, x);
  return contaRic(l->next, x);
}

void stampaInversaRic(TipoLista l) {
  if (l != NULL) {
    stampaInversaRic(l->next);        /* prima il resto (al contrario) */
    printf("%d ", l->info);           /* poi questo nodo               */
  }
}
```

Esecuzione simulata di `contaRic(l, 1)` su `l: 1 -> 4 -> 1 -> NULL`:

```
RDA contaRic  l=NULL          caso base                     -> 0
RDA contaRic  l=[1]->NULL     info==1:  1 + contaRic(NULL)  -> 1 + 0 = 1
RDA contaRic  l=[4]->...      info!=1:  contaRic([1])       -> 1
RDA contaRic  l=[1]->...      info==1:  1 + contaRic([4])   -> 1 + 1 = 2
RDA main                      riceve 2
```

Il programma di prova completo

Tutte le funzioni di questo capitolo stanno in `liste.c`, con questo `main` di prova:

codice/liste.c

```c
int main(void) {
  TipoLista l = NULL;              /* SEMPRE inizializzare a NULL!      */
  int i, m;

  for (i = 1; i <= 3; i++)
    insTesta(&l, i);               /* passo l'INDIRIZZO di l            */
  stampaLista(l);                  /* 3 -> 2 -> 1 -> NULL               */

  insCoda(&l, 10);
  insCodaRG(&l, 20);
  stampaLista(l);                  /* 3 -> 2 -> 1 -> 10 -> 20 -> NULL   */
  printf("lunghezza: %d (ricorsiva: %d)\n", lunghezza(l), lunghezzaRic(l));

  elimina(&l, 3);                  /* elimino il primo nodo             */
  elimina(&l, 10);                 /* elimino un nodo in mezzo          */
  elimina(&l, 99);                 /* non c'e`: restituisce 0           */
  stampaLista(l);                  /* 2 -> 1 -> 20 -> NULL              */

  if (maxLista(l, &m))
    printf("massimo: %d\n", m);
  printf("contrario: ");
  stampaInversaRic(l);
  printf("\n");
  dealloca(&l);

  insOrdinato(&l, 5); insOrdinato(&l, 1); insOrdinato(&l, 9);
  insOrdinato(&l, 7); insOrdinato(&l, 1);
  stampaLista(l);                  /* 1 -> 1 -> 5 -> 7 -> 9 -> NULL     */
  printf("quanti 1: %d\n", contaRic(l, 1));
  printf("cerco 7: %s\n", cerca(l, 7) != NULL ? "trovato" : "assente");
  dealloca(&l);
  printf("dopo dealloca: ");
  stampaLista(l);                  /* NULL                              */
  return 0;
}
```

Output reale:

```
3 -> 2 -> 1 -> NULL
3 -> 2 -> 1 -> 10 -> 20 -> NULL
lunghezza: 5 (ricorsiva: 5)
2 -> 1 -> 20 -> NULL
massimo: 20
contrario: 20 1 2
1 -> 1 -> 5 -> 7 -> 9 -> NULL
quanti 1: 2
cerco 7: trovato
dopo dealloca: NULL
```

Liste di struct (come all'esame)

All'esame l'informazione del nodo non è un `int` ma una **struct** con i dati dell'elemento. Lo schema è sempre questo:

```
typedef struct {
  char *nome;                  /* stringa di lunghezza ignota: esatta   */
  char codice[9 + 1];          /* stringa di 9 caratteri                */
  int quantita;
  char tipo;
} TipoElemento;

struct nodoElem {
  TipoElemento info;
  struct nodoElem *next;
};

typedef struct nodoElem TipoNodoElem;
typedef TipoNodoElem *TipoListaElem;
```

L'accesso ai campi diventa `p->info.quantita`, `p->info.nome`, `strcmp(p->info.codice, cod)`.

ESERCIZIO 14.1

Scrivi `int sommaPari(TipoLista l)` (somma dei valori pari) in versione iterativa e ricorsiva.

Soluzione

```
int sommaPari(TipoLista l) {
  int s = 0;
  for (; l != NULL; l = l->next)
    if (l->info % 2 == 0)
      s += l->info;
  return s;
}

int sommaPariRic(TipoLista l) {
  if (l == NULL)
    return 0;
  if (l->info % 2 == 0)
    return l->info + sommaPariRic(l->next);
  return sommaPariRic(l->next);
}
```

ESERCIZIO 14.2

Scrivi `int eliminaUltimo(TipoLista *pl)` che elimina l'ultimo nodo e restituisce 1, oppure 0 se la lista è vuota.

Soluzione

```c
int eliminaUltimo(TipoLista *pl) {
  TipoNodo *prec = NULL, *corr = *pl;
  if (corr == NULL)
    return 0;
  while (corr->next != NULL) {     /* mi fermo sull'ultimo */
    prec = corr;
    corr = corr->next;
  }
  if (prec == NULL)                /* c'era un solo nodo   */
    *pl = NULL;
  else
    prec->next = NULL;
  free(corr);
  return 1;
}
```

ESERCIZIO 14.3

Scrivi `TipoLista copiaLista(TipoLista l)` che crea e restituisce una **copia** della lista, con gli elementi nello stesso ordine.

Soluzione

Versione ricorsiva (la più semplice): la copia di una lista vuota è vuota; altrimenti creo un nodo con la stessa info e gli attacco la copia del resto.

```
TipoLista copiaLista(TipoLista l) {
  TipoNodo *n;
  if (l == NULL)
    return NULL;
  n = malloc(sizeof(TipoNodo));
  if (n == NULL)
    return NULL;
  n->info = l->info;
  n->next = copiaLista(l->next);
  return n;
}
```

ESERCIZIO 14.4

Scrivi `void invertiLista(TipoLista *pl)` che inverte la lista **senza allocare** nuovi nodi (cambiando solo i `next`).

Soluzione

```
void invertiLista(TipoLista *pl) {
  TipoNodo *prec = NULL, *corr = *pl, *succ;
  while (corr != NULL) {
    succ = corr->next;     /* salvo il resto della lista     */
    corr->next = prec;     /* giro la freccia                */
    prec = corr;           /* avanzo i due puntatori         */
    corr = succ;
  }
  *pl = prec;              /* l'ultimo nodo e` la nuova testa */
}
```

CAPITOLO 15

Alberi binari (cenni)

(slide e EG di fine corso; testo2 cap. 8, "limitatamente a quanto visto a lezione")

Il programma prevede gli alberi come **cenni**. Nei testi d'esame per gli immatricolati dal 2025/26 non compaiono, ma in quelli degli anni precedenti sì (esercizi 2 e 3 della traccia "Giovani Marmotte"). Sappi almeno: definizione, tipi in C, visite ricorsive, funzioni ricorsive semplici.

Definizione

Un **albero binario** è **vuoto**, oppure è formato da un nodo **radice** con due sottoalberi binari: il **sinistro** e il **destro**. È una definizione **ricorsiva**, come quella della lista.

```
            50              <- radice
          /    \
        30      70
       /  \    /  \
     20   40  60   80
         /      \
       35        65          <- foglie: 20, 35, 65, 80
```

Termini: **radice** (il nodo in cima), **figli** e **padre**, **foglia** (nodo senza figli), **altezza** (lunghezza del cammino più lungo dalla radice a una foglia: qui 3).

I tipi in C

codice/alberi.c

```c
typedef int TipoInfo;

struct nodoAlbero {
  TipoInfo info;
  struct nodoAlbero *sinistro;     /* sottoalbero sinistro             */
  struct nodoAlbero *destro;       /* sottoalbero destro               */
};

typedef struct nodoAlbero TipoNodoAlbero;
typedef TipoNodoAlbero *TipoAlbero;   /* albero = puntatore alla radice */
```

Le tre visite (in profondità)

Visitare un albero significa esaminare tutti i suoi nodi. Le tre visite ricorsive differiscono solo per **quando** si esamina la radice:

- **preordine** (anticipata): radice, poi sottoalbero sinistro, poi destro;
- **inordine** (simmetrica): sinistro, radice, destro;

- **postordine** (posticipata): sinistro, destro, radice.

codice/alberi.c

```c
void preordine(TipoAlbero a) {     /* RADICE - sinistro - destro        */
  if (a != NULL) {
    printf("%d ", a->info);
    preordine(a->sinistro);
    preordine(a->destro);
  }
}

void inordine(TipoAlbero a) {      /* sinistro - RADICE - destro        */
  if (a != NULL) {
    inordine(a->sinistro);
    printf("%d ", a->info);
    inordine(a->destro);
  }
}

void postordine(TipoAlbero a) {    /* sinistro - destro - RADICE        */
  if (a != NULL) {
    postordine(a->sinistro);
    postordine(a->destro);
    printf("%d ", a->info);
  }
}
```

Sull'albero disegnato sopra:

```
preordine:  50 30 20 40 35 70 60 65 80
inordine:   20 30 35 40 50 60 65 70 80     <- ordinato! (e` un ABR)
postordine: 20 35 40 30 65 60 80 70 50
```

Esecuzione simulata di una visita (con RDA)

`inordine` sul sottoalbero `30` (figli `20` e `40`, con `40` che ha il figlio sinistro `35`):

```
inordine(30)
 +- inordine(20)
 |   +- inordine(NULL)  -> ritorna subito
 |   +- stampa 20
 |   +- inordine(NULL)  -> ritorna subito
 +- stampa 30
 +- inordine(40)
     +- inordine(35)
     |   +- inordine(NULL); stampa 35; inordine(NULL)
     +- stampa 40
     +- inordine(NULL)
stampato: 20 30 35 40
```

La pila delle RDA, nel momento in cui si stampa 35, contiene (dal basso): `main`, `inordine(50)`, `inordine(30)`, `inordine(40)`, `inordine(35)`.

Funzioni ricorsive sugli alberi

Schema: caso base `a == NULL`; caso ricorsivo che combina la radice con i risultati sui due sottoalberi.

codice/alberi.c

```c
int contaNodi(TipoAlbero a) {
  if (a == NULL)
    return 0;
  return 1 + contaNodi(a->sinistro) + contaNodi(a->destro);
}

int sommaNodi(TipoAlbero a) {
  if (a == NULL)
    return 0;
  return a->info + sommaNodi(a->sinistro) + sommaNodi(a->destro);
}

int altezza(TipoAlbero a) {        /* albero vuoto: -1; solo radice: 0  */
  int hs, hd;
  if (a == NULL)
    return -1;
  hs = altezza(a->sinistro);
  hd = altezza(a->destro);
  return 1 + (hs > hd ? hs : hd);
}

int contaFoglie(TipoAlbero a) {
  if (a == NULL)
    return 0;
  if (a->sinistro == NULL && a->destro == NULL)
    return 1;                      /* foglia: nessun figlio             */
  return contaFoglie(a->sinistro) + contaFoglie(a->destro);
}
```

Albero binario di ricerca (ABR)

Un **ABR** è un albero binario in cui, per ogni nodo, **tutti** i valori nel sottoalbero sinistro sono **minori** della sua info e quelli nel destro sono **maggiori o uguali**. Conseguenze: la **visita inordine dà i valori ordinati**; la **ricerca** scende a sinistra o a destra, come la ricerca binaria (su un albero bilanciato circa $\log_2 n$ passi).

codice/alberi.c

```c
/* Albero Binario di Ricerca (ABR): a sinistra i valori minori della
   radice, a destra quelli maggiori (o uguali).                        */
int insABR(TipoAlbero *pa, TipoInfo x) {
  if (*pa == NULL) {              /* posto trovato: creo la foglia     */
    *pa = malloc(sizeof(TipoNodoAlbero));
    if (*pa == NULL)
      return 0;
    (*pa)->info = x;
    (*pa)->sinistro = NULL;
    (*pa)->destro = NULL;
    return 1;
  }
  if (x < (*pa)->info)
    return insABR(&((*pa)->sinistro), x);
  return insABR(&((*pa)->destro), x);
}

TipoNodoAlbero *cercaABR(TipoAlbero a, TipoInfo x) {
  if (a == NULL || a->info == x)
    return a;
  if (x < a->info)
    return cercaABR(a->sinistro, x);
  return cercaABR(a->destro, x);
}
```

Per **inserire** si passa `TipoAlbero *` perché, nel caso base, bisogna scrivere il nuovo nodo nel puntatore (radice o campo `sinistro` / `destro` del padre).

codice/alberi.c

```c
void deallocaAlbero(TipoAlbero *pa) {   /* in POSTORDINE: prima i figli */
  if (*pa != NULL) {
    deallocaAlbero(&((*pa)->sinistro));
    deallocaAlbero(&((*pa)->destro));
    free(*pa);
    *pa = NULL;
  }
}
```

La deallocazione si fa in **postordine**: prima i figli, poi il nodo (dopo `free` non si potrebbero più raggiungere i figli).

ESERCIZIO 15.1

Scrivi `int contaMaggiori(TipoAlbero a, int x)` che conta i nodi con info > x (albero binario qualsiasi), e `int massimoABR(TipoAlbero a)` che restituisce il massimo di un ABR non vuoto.

Soluzione

```c
int contaMaggiori(TipoAlbero a, int x) {
  if (a == NULL)
    return 0;
  return (a->info > x) + contaMaggiori(a->sinistro, x)
                       + contaMaggiori(a->destro, x);
}

int massimoABR(TipoAlbero a) {   /* in un ABR il massimo e` il nodo piu` a destra */
  while (a->destro != NULL)
    a = a->destro;
  return a->info;
}
```

`(a->info > x)` vale 1 o 0: un modo compatto per contare.

CAPITOLO 16

Il metodo per lo scritto

Qui metto insieme tutto in una procedura da seguire in aula. Leggila prima di ogni traccia di esercitazione, finché non diventa automatica.

Gestione del tempo (2 ore e 30)

Tempo	Cosa fare
0-10 min	Leggere **tutto** il testo. Sottolineare: campi e loro tipi, cosa riceve e cosa restituisce ogni funzione, se la lista è ordinata.
10-25 min	**Tipi** (esercizio 1, prima parte). Sono la base di tutto: se sono sbagliati, sono sbagliati anche gli esercizi 1 e 2.
25-60 min	`eser1`
60-100 min	`eser2`
100-135 min	Esercizio 3
135-150 min	**Rilettura**: punti e virgola, `*` nei parametri di output, `NULL`, `free`, `strcmp`.

Esercizio 1, prima parte: i tipi

Per ogni dato del testo scegli il tipo:

Il testo dice...	In C
"stringa di lunghezza non nota a priori / non prefissata"	`char *campo;` (stringa **esatta**, allocata con `malloc`)
"stringa di k caratteri"	`char campo[k + 1];` (+1 per il `'\0'`)
"un carattere (R o A...)"	`char campo;`
"un intero", "numero di...", "anni"	`int campo;`
"prezzo", "costo", "media reale"	`double campo;`
"data", "ora"	una piccola struct (`struct data { int g, m, a; };`)

Poi lo schema fisso (adatta i nomi al testo):

```
#define LUNGCOD 9

typedef struct {
  /* ...i campi... */
} TipoElemento;

struct nodoElemento {
  TipoElemento info;
  struct nodoElemento *next;
};

typedef struct nodoElemento TipoNodoElemento;
typedef TipoNodoElemento *TipoLista;   /* usa il nome chiesto dal testo! */
```

Aggiungi un commento per ogni campo con il significato: mostra che hai capito il testo.

Esercizio 1, seconda parte: la funzione eser1

1. **Intestazione**. "Restituisca A, B e C": scegli **uno** dei risultati per il `return` (di solito il puntatore o il conteggio) e gli **altri come parametri puntatore**. "Ricevendo (almeno) una lista e un valore": i parametri sono la lista (`TipoLista` , perché la funzione legge soltanto) e il valore.
2. **Inizializza** subito i parametri di output (`*conta = 0;`).
3. **Scansione** con lo schema `for (p = l; p != NULL; p = p->next)` .
4. Dentro: contatori `(*conta)++` , accumulatori, confronti con `strcmp` , ricerca del massimo (il candidato parte dal **primo nodo**, non da 0).
5. **Dopo il ciclo**: le medie (attenzione alla divisione per zero e alla divisione intera), il `return` .
6. Gestisci la **lista vuota** (restituisci `NULL` o 0 e dillo in un commento).

Esercizio 2: modificare la lista

1. Il parametro è `TipoLista *pl` se la testa può cambiare (quasi sempre).
2. Per inserire o eliminare in mezzo serve il **predecessore**: due puntatori `prec` / `corr` , oppure il **record generatore**.
3. Considera **sempre** questi casi e prova mentalmente il codice su ciascuno: - lista **vuota**; - l'elemento interessato è il **primo**; - è l'**ultimo**; - è **in mezzo**; - (per le ricerche) **non c'è**.
4. **Inserimento**: `malloc` del nodo (controlla `NULL`), `malloc` + `strcpy` delle stringhe esatte, riempimento di tutti i campi, aggancio (`nuovo->next = corr; prec->next = nuovo;` oppure nuova testa).
5. **Eliminazione**: scavalca (`prec->next = corr->next;` oppure `*pl = corr->next;`), poi `free` delle stringhe esatte, poi `free` del nodo.
6. Restituisci un esito (1 = fatto, 0 = non fatto) anche se il testo non lo chiede: è robustezza.

Esercizio 3

- **Test (scatola nera / trasparente)**: segui lo schema del capitolo 13 (descrizione del metodo, tabella dei test motivati con dati concreti, esecuzione simulata).
- **Floating point**: i 5 passi del capitolo 2, mostrando **tutti** i calcoli (divisioni, moltiplicazioni, normalizzazione, eccesso 127) e una verifica finale.
- **Funzione ricorsiva con RDA**: caso base + caso ricorsivo, poi la pila delle RDA su un esempio concreto (capitolo 11), con l'andata e il ritorno dei valori.
- **Visite di albero**: definizione delle tre visite, codice, esempio su un albero di almeno 9 nodi, sequenze prodotte, RDA (capitolo 15).

Checklist finale (rileggi così)

GLI ERRORI CHE COSTANO PIÙ PUNTI

- [ ] Stringhe confrontate con `==` invece di `strcmp(...) == 0`, o copiate con `=` invece di `strcpy`.
- [ ] `malloc(strlen(s))` senza il **+1** per il `'\0'`.
- [ ] Parametro di output usato senza `*` (`conta++` invece di `(*conta)++`).
- [ ] Funzione che cambia la testa ma riceve `TipoLista` invece di `TipoLista *`.
- [ ] `p->next` usato quando `p` può essere `NULL` (ordine delle condizioni nei `while`).
- [ ] `free(nodo)` prima di aver salvato o collegato `nodo->next`.
- [ ] Stringa esatta non liberata prima del nodo.
- [ ] Massimo inizializzato a 0 invece che al primo elemento.
- [ ] Media calcolata con divisione intera quando il testo vuole un reale (o viceversa) e divisione per zero.
- [ ] Lista vuota non gestita.
- [ ] Variabili non inizializzate (contatori, `TipoLista l = NULL`).

SCRIVERE BENE CONTA

Il codice su carta viene letto da una persona: indenta, usa nomi chiari (`prec`, `corr`, `nuovo`), scrivi due righe di commento con l'algoritmo prima del codice. Se non ricordi il nome di una funzione di libreria, scrivi un commento che dica cosa fa: è meglio di un'invenzione.

CAPITOLO 17

Le tracce del professore, risolte

Queste sono **le tracce d'esempio pubblicate sul sito del corso** (cartella "Testi d'esame d'esempio"), con soluzioni complete. Il codice è stato compilato e provato. Prima di leggere ogni soluzione, prova a scriverla tu su carta.

Traccia 1: "Segnala un guasto!"

TESTO (ESEMPIO 1, IMMATRICOLATI 2025-26 E SUCCESSIVI; TEMPO: 2H30)

Esercizio 1 (0-13 punti). Una società di distribuzione del gas organizza tutte le segnalazioni di guasto in una tabella di guasti. Ogni guasto è caratterizzato da: **descrizione** (stringa di lunghezza non nota a priori); **tipologia** (intero: 1 = perdita lieve, 2 = interruzione, 3 = perdita ragguardevole); **servizio** (carattere: R per "riscaldamento + acqua calda", A per "solo acqua calda"); **codice utente** (stringa di 9 caratteri alfanumerici); **numero d'ordine** della chiamata (intero). Quando una squadra è disponibile le si assegna il guasto più importante, eliminandolo dalla tabella. **importanza = tipologia*11 - numero d'ordine**. Scrivere la definizione dei tipi necessari a gestire la tabella di guasti come **lista** rappresentata mediante record e puntatori. Poi scrivere una funzione `eser1` che, ricevendo (almeno) una lista di guasti e un intero `imp`, restituisca il **puntatore alla segnalazione più importante**, il **numero di guasti più importanti di imp** e il **numero di guasti meno importanti di imp**.

Esercizio 2 (10 punti). Scrivere una funzione `eser2()` che, ricevendo (almeno) una lista di segnalazioni, **elimini la segnalazione più importante**.

Esercizio 3 (da -1 a 7 punti). Descrivere con completezza il metodo di verifica della correttezza **a scatola trasparente**. Applicarlo alla verifica di `eser1()`: quali test e con quali dati di prova? Giustificare ciascun test. Verificare il successo dei test con esecuzioni simulate. *(Nella versione per gli immatricolati 2024-25 e precedenti l'esercizio 3 chiedeva invece il metodo a **scatola nera**: lo risolvo sotto.)*

Esercizio 1: i tipi

codice/guasti.c

```c
#define LUNGCOD 9                  /* codice utente: 9 caratteri         */

typedef struct {
  char *descrizione;               /* stringa "esatta" (lunghezza ignota) */
  int tipologia;                   /* 1 lieve, 2 interruzione, 3 ragguard. */
  char servizio;                   /* 'R' oppure 'A'                      */
  char codiceUtente[LUNGCOD + 1];  /* 9 caratteri + '\0'                  */
  int numOrdine;                   /* numero d'ordine della chiamata      */
} TipoGuasto;

struct nodoGuasto {
  TipoGuasto info;
  struct nodoGuasto *next;
};

typedef struct nodoGuasto TipoNodoGuasto;
typedef TipoNodoGuasto *TipoListaGuasti;   /* la "tabella" dei guasti   */
```

Una funzione di servizio rende il codice più leggibile:

codice/guasti.c

```c
int importanza(TipoGuasto g) {
  return g.tipologia * 11 - g.numOrdine;
}
```

Esercizio 1: la funzione eser1

Tre risultati: il puntatore con `return`, i due conteggi come **parametri di output**. La lista si legge soltanto, quindi si passa come `TipoListaGuasti`.

codice/guasti.c

```c
/* Restituisce il puntatore al nodo col guasto piu` importante (NULL se la
   lista e` vuota). Tramite i parametri di output *piuImp e *menoImp
   restituisce quanti guasti hanno importanza > imp e < imp.            */
TipoNodoGuasto *eser1(TipoListaGuasti lista, int imp, int *piuImp, int *menoImp) {
  TipoNodoGuasto *max = lista;     /* candidato: il primo (o NULL)      */
  TipoNodoGuasto *p;
  int i;

  *piuImp = 0;                     /* inizializzo i contatori di output */
  *menoImp = 0;
  for (p = lista; p != NULL; p = p->next) {
    i = importanza(p->info);
    if (i > imp)
      (*piuImp)++;                 /* attenzione alle parentesi!        */
    else if (i < imp)
      (*menoImp)++;
    if (i > importanza(max->info)) /* qui max != NULL perche' p != NULL */
      max = p;
  }
  return max;
}
```

Algoritmo: scansione unica; per ogni nodo calcolo l'importanza, aggiorno uno dei due contatori (un guasto con importanza **uguale** a `imp` non va in nessuno dei due) e aggiorno il candidato massimo. Con lista vuota il ciclo non viene eseguito e si restituisce `NULL` con entrambi i contatori a 0.

Esercizio 2: eser2

Per eliminare il nodo più importante serve **il suo predecessore**: durante la scansione tengo `prec` / `corr` e, quando trovo un nuovo massimo, ricordo anche il suo predecessore (`precMax`).

codice/guasti.c

```c
/* Elimina dalla lista il guasto piu` importante.
   Restituisce 1 se ha eliminato qualcosa, 0 se la lista era vuota.     */
int eser2(TipoListaGuasti *plista) {
  TipoNodoGuasto *prec, *corr;     /* scansione                          */
  TipoNodoGuasto *precMax, *max;   /* il massimo e il suo predecessore   */

  if (*plista == NULL)
    return 0;

  precMax = NULL;
  max = *plista;
  prec = *plista;
  corr = (*plista)->next;
  while (corr != NULL) {
    if (importanza(corr->info) > importanza(max->info)) {
      max = corr;
      precMax = prec;
    }
    prec = corr;
    corr = corr->next;
  }

  if (precMax == NULL)             /* il massimo e` il primo nodo        */
    *plista = max->next;
  else
    precMax->next = max->next;     /* scavalco il nodo da eliminare      */

  free(max->info.descrizione);     /* prima la stringa allocata ...      */
  free(max);                       /* ... poi il nodo                    */
  return 1;
}
```

Casi controllati: lista vuota (restituisce 0); massimo in testa (`precMax == NULL`: cambia la testa, per questo il parametro è `TipoListaGuasti *`); massimo in mezzo o in coda (scavalcamento). Prima del nodo si libera la descrizione, che è una stringa esatta.

Per provare le funzioni serve anche un inserimento (non richiesto dal testo), che mostra come si riempie un nodo con una stringa esatta:

codice/guasti.c

```c
/* aggiunge in testa un guasto; la descrizione viene duplicata "esattamente" */
int aggiungiGuasto(TipoListaGuasti *plista, char *descr, int tipo, char serv,
                   char *cod, int ordine) {
  TipoNodoGuasto *nuovo = malloc(sizeof(TipoNodoGuasto));
  if (nuovo == NULL)
    return 0;
  nuovo->info.descrizione = malloc(strlen(descr) + 1);   /* esatta!     */
  if (nuovo->info.descrizione == NULL) {
    free(nuovo);
    return 0;
  }
  strcpy(nuovo->info.descrizione, descr);
  nuovo->info.tipologia = tipo;
  nuovo->info.servizio = serv;
  strncpy(nuovo->info.codiceUtente, cod, LUNGCOD);
  nuovo->info.codiceUtente[LUNGCOD] = '\0';
  nuovo->info.numOrdine = ordine;
  nuovo->next = *plista;
  *plista = nuovo;
  return 1;
}
```

Output della prova (`guasti.c`): con i guasti di importanza 18, 30, 20, 10 e `imp = 18` , `eser1` restituisce il guasto "odore di gas in cantina" (30), 2 più importanti, 1 meno importante; tre chiamate di `eser2` eliminano, nell'ordine, quello da 30 (in mezzo), quello da 20 (in mezzo) e quello da 18 (in testa).

Esercizio 3: scatola trasparente applicata a eser1

Il metodo. Nella verifica a scatola trasparente i test si scelgono **guardando il codice** della funzione, con l'obiettivo di far eseguire, nel complesso dei test, tutte le sue parti. Si applicano criteri di copertura via via più scrupolosi: (1) **copertura delle istruzioni**: ogni istruzione eseguita almeno una volta; (2) **copertura delle decisioni**: ogni decisione (condizione di `if` / `for` / `while`) almeno una volta vera e una volta falsa; (3) **copertura delle condizioni**: ogni condizione elementare di una decisione composta almeno una volta vera e una volta falsa; (4) **valori limite**: test con valori uguali ai limiti delle condizioni e vicini a essi. Per i cicli si provano 0, 1 e più iterazioni. Per ogni test si calcola prima l'**output atteso**, poi si esegue e lo si confronta con l'**output effettivo**; se differiscono si fa debugging. Nessun criterio garantisce l'assenza di errori, ma più criteri si soddisfano più la verifica è affidabile.

Le decisioni in eser1:

- **D1** `p != NULL` (ciclo `for`);
- **D2** `i > imp` ;
- **D3** `i < imp` (valutata solo se D2 è falsa);
- **D4** `i > importanza(max->info)` .

I test (la lista è scritta dalla testa alla coda; per ogni guasto indico tipologia/numero d'ordine e l'importanza risultante):

Test	Lista (importanze)	imp	Perché	Atteso
T1	vuota	0	D1 falsa subito (0 iterazioni); istruzioni fuori dal ciclo	`NULL`, 0, 0
T2	g(2/2) → **20**	20	1 iterazione; D2 e D3 false (valore **limite** i == imp); D4 falsa	g, 0, 0
T3	g(1/1)=**10**, g(3/3)=**30**, g(2/4)=**18**	18	più iterazioni; D2 vera (30) e falsa; D3 vera (10) e falsa (18 == imp); D4 vera (30 > 10) e falsa (18)	nodo da 30, 1, 1
T4	**30**, 10, 18	25	massimo **in testa**: D4 sempre falsa, `max` deve restare il primo	nodo da 30, 1, 2
T5	10, 18, **30**	9	massimo **in coda**; tutti più importanti	nodo da 30, 3, 0

T1 e T3 da soli coprono già tutte le istruzioni e tutte le decisioni (vere e false); T2, T4 e T5 aggiungono i valori limite (importanza uguale a `imp`, un solo nodo, massimo agli estremi).

Esecuzione simulata di T3 (imp = 18):

passo	p (importanza)	i	D2 i>18	D3 i<18	*piuImp	*menoImp	D4	max
inizio	-	-	-	-	0	0	-	nodo 10
1	nodo 10	10	F	**V**	0	1	10>10 F	nodo 10
2	nodo 30	30	**V**	-	1	1	30>10 **V**	nodo 30
3	nodo 18	18	F	F	1	1	18>30 F	nodo 30
fine	NULL (D1 falsa)				**1**	**1**		**nodo 30**

Output effettivo = atteso. Analogamente T1: il ciclo non parte, i contatori restano 0 e `max` vale `lista` cioè `NULL`: corretto.

Variante: scatola nera applicata a eser1

Il metodo. I test si scelgono **senza guardare il codice**, in base alla **specifica**: si partiziona l'insieme dei possibili input in **classi di equivalenza** (input per cui ci si aspetta lo stesso comportamento) e si sceglie un rappresentante per ogni classe. Più le classi sono specifiche (casi particolari, casi limite), più test si fanno e più si è sicuri.

Classi per eser1 (input: una lista di guasti e un intero imp):

Classe	Rappresentante	Atteso
lista vuota	`NULL`, imp = 5	`NULL`, 0, 0
un solo guasto, più importante di imp	[20], imp = 5	g20, 1, 0
un solo guasto, meno importante	[20], imp = 25	g20, 0, 1
un solo guasto, importanza = imp	[20], imp = 20	g20, 0, 0
più guasti, massimo in testa	[30, 10, 18], imp = 15	g30, 2, 1
più guasti, massimo in coda	[10, 18, 30], imp = 15	g30, 2, 1
più guasti, massimo in mezzo	[10, 30, 18], imp = 15	g30, 2, 1
tutti più importanti di imp	[10, 30, 18], imp = 0	g30, 3, 0
tutti meno importanti	[10, 30, 18], imp = 40	g30, 0, 3
due guasti con la stessa importanza massima	[30, 30]	il primo dei due (la specifica non lo dice: va deciso e scritto)
importanze negative (numero d'ordine grande)	tipologia 1, ordine 50 → -39	il massimo è comunque il più grande

L'ultima classe è importante: un'implementazione che partisse con "massimo = 0" sbaglierebbe con importanze tutte negative.

Traccia 2: "Giovani Marmotte"

TESTO (ESEMPI 2 E 3, IMMATRICOLATI 2025-26 E SUCCESSIVI)

Esercizio 1 (0-13 punti). Il reggimento delle Giovani Marmotte è composto da iscritti, ciascuno specificato da: **cognome e nome** (stringa di lunghezza non prefissata); **associazione** (numero di anni di presenza nel reggimento); **grado** (stringa di 8 caratteri); **numero medaglie** ricevute durante l'anno; **numero reprimende**. Scrivere la definizione del tipo `TipoReggimento`, che rappresenta il reggimento come **lista** (record e puntatori). Poi scrivere una funzione `eser1` che, ricevendo (almeno) un reggimento `gm` e un grado `rank`, restituisca il **numero di marmotte di quel grado**, la **media (intera) delle loro medaglie** e la **media (intera) delle loro reprimende**.

Esercizio 2 (10 punti). Assumendo la lista ordinata in senso crescente del cognome e nome, scrivere `eser2()` che, ricevendo una lista-reggimento e 5 ulteriori parametri con i dati di una nuova marmotta, la **aggiunga mantenendo l'ordinamento**.

Esercizio 3 (da -1 a 7 punti). *Esempio 2:* rappresentazione floating point su 32 bit di **1216.37**, descrivendo i passaggi. *Esempio 3:* funzione **ricorsiva** `eser3()` che, ricevendo un reggimento e un grado, restituisca il numero di marmotte con quel grado; mostrarne il funzionamento su un reggimento di almeno 5 marmotte con un'**esecuzione simulata** dettagliando i **record di attivazione**.

Esercizio 1

codice/marmotte.c

```c
#define LUNGGRADO 8

typedef struct {
  char *cognomeNome;              /* stringa esatta                    */
  int associazione;               /* anni di presenza nel reggimento   */
  char grado[LUNGGRADO + 1];      /* 8 caratteri + '\0'                */
  int medaglie;
  int reprimende;
} TipoMarmotta;

struct nodoMarmotta {
  TipoMarmotta info;
  struct nodoMarmotta *next;
};

typedef struct nodoMarmotta TipoNodoMarmotta;
typedef TipoNodoMarmotta *TipoReggimento;
```

codice/marmotte.c

```c
/* Restituisce quante marmotte hanno grado rank; in *mediaMed e *mediaRep
   le medie (intere) di medaglie e reprimende di quelle marmotte.
   Se non ce ne sono, le medie valgono 0 (evito la divisione per zero). */
int eser1(TipoReggimento gm, char *rank, int *mediaMed, int *mediaRep) {
  int quante = 0, sommaMed = 0, sommaRep = 0;

  while (gm != NULL) {
    if (strcmp(gm->info.grado, rank) == 0) {
      quante++;
      sommaMed += gm->info.medaglie;
      sommaRep += gm->info.reprimende;
    }
    gm = gm->next;
  }
  if (quante > 0) {
    *mediaMed = sommaMed / quante;     /* divisione intera: media intera */
    *mediaRep = sommaRep / quante;
  } else {
    *mediaMed = 0;
    *mediaRep = 0;
  }
  return quante;
}
```

"Media intera" significa **divisione intera** tra interi (niente cast). Senza marmotte di quel grado si evita la divisione per zero.

Esercizio 2

codice/marmotte.c

```c
/* Inserisce una nuova marmotta mantenendo l'ordine crescente di cognomeNome.
   Restituisce 1 se l'inserimento riesce, 0 se manca memoria.           */
int eser2(TipoReggimento *pgm, char *cn, int ass, char *grado,
          int med, int rep) {
  TipoNodoMarmotta *nuovo, *prec, *corr;

  /* 1. creo e riempio il nuovo nodo */
  nuovo = malloc(sizeof(TipoNodoMarmotta));
  if (nuovo == NULL)
    return 0;
  nuovo->info.cognomeNome = malloc(strlen(cn) + 1);
  if (nuovo->info.cognomeNome == NULL) {
    free(nuovo);
    return 0;
  }
  strcpy(nuovo->info.cognomeNome, cn);
  nuovo->info.associazione = ass;
  strncpy(nuovo->info.grado, grado, LUNGGRADO);
  nuovo->info.grado[LUNGGRADO] = '\0';
  nuovo->info.medaglie = med;
  nuovo->info.reprimende = rep;

  /* 2. cerco la posizione: il primo nodo con cognomeNome >= cn */
  prec = NULL;
  corr = *pgm;
  while (corr != NULL && strcmp(corr->info.cognomeNome, cn) < 0) {
    prec = corr;
    corr = corr->next;
  }

  /* 3. aggancio */
  nuovo->next = corr;
  if (prec == NULL)
    *pgm = nuovo;                  /* nuova testa                       */
  else
    prec->next = nuovo;
  return 1;
}
```

I cinque parametri sono i cinque campi. La posizione è il primo nodo con nome **maggiore o uguale** a quello nuovo (`strcmp(...) < 0` fa avanzare).

Esercizio 3 (esempio 2): floating point

È svolto per intero, con tutti i passaggi, nel capitolo 2. Il risultato è

```
0 10001001 00110000000101111010111
```

(segno 0; esponente 10 + 127 = 137 = `10001001`; mantissa = le cifre dopo l'1 di `1.0011000000 0101111010111`).

Esercizio 3 (esempio 3): ricorsione con RDA

codice/marmotte.c

```c
/* Versione ricorsiva: quante marmotte di grado rank nel reggimento gm */
int eser3(TipoReggimento gm, char *rank) {
  if (gm == NULL)                                  /* caso base         */
    return 0;
  if (strcmp(gm->info.grado, rank) == 0)           /* caso ricorsivo 1  */
    return 1 + eser3(gm->next, rank);
  return eser3(gm->next, rank);                    /* caso ricorsivo 2  */
}
```

Esecuzione simulata sul reggimento (in ordine di cognome):

```
gm --> [Amato Sara | general5] --> [Bianchi Ugo | berretto] --> [Conti Leo | berretto]
   --> [Neri Luca | berretto] --> [Rossi Anna | marmotta] --> NULL
```

chiamata dal main: `n = eser3(gm, "berretto");`

```
FASE DI ANDATA: ogni RDA contiene i parametri (gm = indirizzo del nodo, rank)
e il punto di ritorno

 +-------------------------------------------------------------+
 | RDA 6: gm = NULL                 caso base        -> 0       |  in cima
 +-------------------------------------------------------------+
 | RDA 5: gm = [Rossi, marmotta]    grado != rank:  eser3(NULL) |
 +-------------------------------------------------------------+
 | RDA 4: gm = [Neri, berretto]     uguale: 1 + eser3([Rossi])  |
 +-------------------------------------------------------------+
 | RDA 3: gm = [Conti, berretto]    uguale: 1 + eser3([Neri])   |
 +-------------------------------------------------------------+
 | RDA 2: gm = [Bianchi, berretto]  uguale: 1 + eser3([Conti])  |
 +-------------------------------------------------------------+
 | RDA 1: gm = [Amato, general5]    diverso: eser3([Bianchi])   |
 +-------------------------------------------------------------+
 | RDA main: n = ?                                              |
 +-------------------------------------------------------------+
```

```
FASE DI RITORNO (le RDA si chiudono dall'alto verso il basso)

RDA 6 restituisce 0
RDA 5 restituisce 0              (valore di eser3(NULL))
RDA 4 restituisce 1 + 0 = 1
RDA 3 restituisce 1 + 1 = 2
RDA 2 restituisce 1 + 2 = 3
RDA 1 restituisce 3              (valore di eser3([Bianchi]))
main:  n = 3
```

In ogni RDA `rank` punta sempre alla stessa stringa `"berretto"` del chiamante (si copia il puntatore, non la stringa).

Traccia 3 (programma degli anni precedenti): "Marmotte sull'albero"

TESTO (ESEMPIO 2, IMMATRICOLATI 2024-25 E PRECEDENTI)

Stessi dati della traccia 2. I gradi possibili sono in un array globale di stringhe `gradi` ={"semplice", "marmotta", "berretto", "general0", ..., "general9", "mogol-00", ..., "mogol-99", "GR.mogol"}. A fine anno, per ogni medaglia la marmotta **sale** di un grado, per ogni reprimenda **scende** di un grado, senza superare "GR.mogol" né scendere sotto "semplice". **Esercizio 2**: definire la struttura dati per rappresentare il reggimento come **albero**; scrivere `eser2()` che, ricevendo un albero del reggimento e un cognome e nome, aggiorni il record della marmotta (se c'è) con il bilancio delle medaglie e delle reprimende. **Esercizio 3**: descrivere formalmente gli algoritmi di **visita** di un albero; esemplificarli su un albero di interi di almeno 9 nodi, con i record di attivazione.

Scelgo un **albero binario di ricerca** ordinato per cognome e nome: la ricerca della marmotta scende a sinistra o a destra.

codice/marmotte_albero.c

```c
typedef struct {
  char *cognomeNome;
  int associazione;
  char grado[LUNGGRADO + 1];
  int medaglie;
  int reprimende;
} TipoMarmotta;

struct nodoAlbMarmotta {
  TipoMarmotta info;
  struct nodoAlbMarmotta *sinistro, *destro;
};

typedef struct nodoAlbMarmotta TipoNodoAlbMarmotta;
typedef TipoNodoAlbMarmotta *TipoAlberoReggimento;
```

codice/marmotte_albero.c

```c
/* posizione di un grado nell'array gradi (-1 se non c'e`) */
int indiceGrado(char *g) {
  int i;
  for (i = 0; i < NUMGRADI; i++)
    if (strcmp(gradi[i], g) == 0)
      return i;
  return -1;
}

/* cerca la marmotta cn nell'ABR e ne aggiorna il grado col bilancio
   medaglie - reprimende; restituisce 1 se trovata, 0 altrimenti        */
int eser2(TipoAlberoReggimento a, char *cn) {
  int c, pos;
  if (a == NULL)
    return 0;                                   /* non c'e`             */
  c = strcmp(cn, a->info.cognomeNome);
  if (c < 0)
    return eser2(a->sinistro, cn);              /* cerco a sinistra     */
  if (c > 0)
    return eser2(a->destro, cn);                /* cerco a destra       */

  /* trovata: calcolo il nuovo grado */
  pos = indiceGrado(a->info.grado);
  if (pos == -1)
    return 0;                                   /* grado non valido     */
  pos = pos + a->info.medaglie - a->info.reprimende;
  if (pos < 0)
    pos = 0;                                    /* non sotto "semplice" */
  if (pos > NUMGRADI - 1)
    pos = NUMGRADI - 1;                         /* non oltre "GR.mogol" */
  strcpy(a->info.grado, gradi[pos]);
  a->info.medaglie = 0;                         /* bilancio fatto       */
  a->info.reprimende = 0;
  return 1;
}
```

L'esercizio 3 è svolto nel capitolo 15 (tre visite, albero di 9 nodi, sequenze prodotte e pila delle RDA).

CAPITOLO 18

Tracce di esercitazione (nuove, stile esame)

Le ho scritte sul modello delle tracce del professore. **Usale come simulazione d'esame**: carta e penna, 2 ore e 30, niente fascicolo aperto. Poi correggi con le soluzioni e prova il codice al computer.

Traccia A: "Biblioteca di quartiere"

TESTO

Esercizio 1 (0-13 punti). Una biblioteca di quartiere tiene il catalogo dei libri. Ogni libro ha: **titolo** (stringa di lunghezza non nota a priori); **ISBN** (stringa di 13 caratteri); **anno** di pubblicazione (intero); **copie** disponibili (intero); **genere** (carattere: R romanzo, S saggio, F fumetto). Definire i tipi per rappresentare il catalogo come lista (record e puntatori). Scrivere `eser1` che, ricevendo (almeno) il catalogo e un anno, restituisca il **puntatore al libro più vecchio**, il **numero di libri pubblicati prima di quell'anno** e il **totale delle copie disponibili**.

Esercizio 2 (10 punti). Scrivere `eser2` che **elimini dal catalogo tutti i libri con zero copie disponibili** e restituisca quanti libri ha eliminato.

Esercizio 3 (da -1 a 7 punti). Rappresentare in floating point su 32 bit il numero **-45.625**, descrivendo tutti i passaggi. Poi rappresentare in complemento a due con il minimo numero di bit **-300** e **125**, e calcolarne la somma su 16 bit.

Soluzione

Tipi

codice/biblioteca.c

```c
#define LUNGISBN 13

typedef struct {
  char *titolo;                    /* stringa esatta                    */
  char isbn[LUNGISBN + 1];
  int anno;
  int copie;                       /* copie disponibili                 */
  char genere;                     /* 'R' romanzo, 'S' saggio, 'F' fumetto */
} TipoLibro;

struct nodoLibro {
  TipoLibro info;
  struct nodoLibro *next;
};

typedef struct nodoLibro TipoNodoLibro;
typedef TipoNodoLibro *TipoCatalogo;
```

eser1

codice/biblioteca.c

```c
/* restituisce il puntatore al libro piu` vecchio (NULL se catalogo vuoto);
   in *prima: quanti libri sono stati pubblicati prima di anno;
   in *totCopie: il totale delle copie disponibili                       */
TipoNodoLibro *eser1(TipoCatalogo c, int anno, int *prima, int *totCopie) {
  TipoNodoLibro *vecchio = c;
  *prima = 0;
  *totCopie = 0;
  while (c != NULL) {
    if (c->info.anno < anno)
      (*prima)++;
    *totCopie += c->info.copie;
    if (c->info.anno < vecchio->info.anno)
      vecchio = c;
    c = c->next;
  }
  return vecchio;
}
```

eser2: eliminare **più** nodi è più delicato che eliminarne uno. Con il record generatore ogni nodo ha un predecessore; quando elimino `prec->next` **non avanzo** `prec`, perché il nuovo `prec->next` deve ancora essere controllato (potrebbero esserci due libri da eliminare consecutivi).

codice/biblioteca.c

```c
/* elimina TUTTI i libri con 0 copie; restituisce quanti ne ha eliminati.
   Uso il record generatore: cosi` anche il primo nodo ha un "precedente". */
int eser2(TipoCatalogo *pc) {
  TipoNodoLibro rg, *prec, *daEliminare;
  int eliminati = 0;

  rg.next = *pc;
  prec = &rg;
  while (prec->next != NULL) {
    if (prec->next->info.copie == 0) {
      daEliminare = prec->next;
      prec->next = daEliminare->next;   /* scavalco                    */
      free(daEliminare->info.titolo);
      free(daEliminare);
      eliminati++;                       /* NON avanzo prec: il nuovo    */
    } else                               /* prec->next va ancora controllato */
      prec = prec->next;
  }
  *pc = rg.next;
  return eliminati;
}
```

Prova (`biblioteca.c`): con 5 libri, di cui 3 con zero copie (in testa, in mezzo e in coda), `eser2` restituisce 3 e restano i 2 libri con copie.

Esercizio 3

Floating point di -45.625: segno negativo, s = 1. 45 = `101101` (32+8+4+1); 0.625: 0.625×2 = 1.25 → 1; 0.25×2 = 0.5 → 0; 0.5×2 = 1.0 → 1; quindi `.101` . Numero: `101101.101` = `1.01101101` × 2^5. E = 5 + 127 = 132 = `10000100` . M = `01101101` + 15 zeri.

```
-45.625 = 1 10000100 01101101000000000000000
```

Verifica: 1.01101101 × 2^5 = 101101.101 = 45.625, col segno meno. La rappresentazione è **esatta** (la parte frazionaria finisce).

Complemento a due: -300 sta in [-512, 511], quindi servono **10 bit**: 300 = `0100101100` , invertito `1011010011` , +1 = `1011010100` . 125 sta in [-128, 127]: **8 bit**, `01111101` . Somma su 16 bit (estensione del segno): `1111111011010100` + `0000000001111101` = `1111111101010001` , che è -175 (2^{16} - 175 = 65361 = `1111111101010001`). Ok.

Traccia B: "Officina"

TESTO

Esercizio 1 (0-13 punti). Un'officina registra le riparazioni. Ogni riparazione ha: **targa** (stringa di 7 caratteri); **descrizione** del lavoro (stringa di lunghezza non nota a priori); **costo** (reale); **ore** di lavoro (intero); **stato** (carattere: A aperta, C chiusa). Definire i tipi per una lista di riparazioni. Scrivere `eser1` che, ricevendo (almeno) la lista, restituisca il **numero di riparazioni aperte**, il **costo totale** delle riparazioni aperte e la **media (reale) delle ore** delle riparazioni aperte.

Esercizio 2 (10 punti). Scrivere `eser2` che, ricevendo la lista e i dati di una nuova riparazione (targa, descrizione, costo, ore), la **aggiunga in coda** come riparazione aperta, **ma solo se** per quella targa non c'è già una riparazione aperta. Restituire 1 se aggiunta, 0 se già presente.

Esercizio 3 (da -1 a 7 punti). Scrivere una funzione **ricorsiva** `eser3` che restituisca il costo totale di tutte le riparazioni (aperte e chiuse) di una targa data. Mostrare un'esecuzione simulata con i record di attivazione su una lista di almeno 4 riparazioni.

Soluzione

Tipi

codice/officina.c

```c
#define LUNGTARGA 7

typedef struct {
  char targa[LUNGTARGA + 1];
  char *descrizione;              /* stringa esatta                  */
  double costo;
  int ore;
  char stato;                     /* 'A' aperta, 'C' chiusa          */
} TipoRiparazione;

struct nodoRip {
  TipoRiparazione info;
  struct nodoRip *next;
};

typedef struct nodoRip TipoNodoRip;
typedef TipoNodoRip *TipoListaRip;
```

eser1: un risultato col `return`, due come parametri di output. Media **reale**: serve il cast.

codice/officina.c

```c
/* restituisce il numero di riparazioni aperte; in *costoAperte il loro
   costo totale e in *oreMedie la media (reale) delle loro ore          */
int eser1(TipoListaRip l, double *costoAperte, double *oreMedie) {
  int aperte = 0, oreTot = 0;
  *costoAperte = 0;
  for (; l != NULL; l = l->next)
    if (l->info.stato == 'A') {
      aperte++;
      *costoAperte += l->info.costo;
      oreTot += l->info.ore;
    }
  if (aperte > 0)
    *oreMedie = (double) oreTot / aperte;
  else
    *oreMedie = 0;
  return aperte;
}
```

eser2: con il record generatore si scorre fino all'ultimo nodo e, **nello stesso ciclo**, si controlla se la targa ha già una riparazione aperta. Il controllo va fatto **prima** di allocare, così in caso di duplicato non c'è niente da liberare.

codice/officina.c

```c
/* aggiunge in CODA una nuova riparazione aperta, ma solo se per quella
   targa non c'e` gia` una riparazione aperta.
   Restituisce 1 se aggiunta, 0 se gia` presente, -1 se manca memoria.   */
int eser2(TipoListaRip *pl, char *targa, char *descr, double costo, int ore) {
  TipoNodoRip rg, *p, *nuovo;

  rg.next = *pl;
  p = &rg;
  while (p->next != NULL) {        /* scorro fino all'ultimo nodo e     */
    if (strcmp(p->next->info.targa, targa) == 0 && p->next->info.stato == 'A')
      return 0;                    /* intanto controllo i duplicati     */
    p = p->next;
  }
  nuovo = malloc(sizeof(TipoNodoRip));
  if (nuovo == NULL)
    return -1;
  nuovo->info.descrizione = malloc(strlen(descr) + 1);
  if (nuovo->info.descrizione == NULL) {
    free(nuovo);
    return -1;
  }
  strcpy(nuovo->info.descrizione, descr);
  strncpy(nuovo->info.targa, targa, LUNGTARGA);
  nuovo->info.targa[LUNGTARGA] = '\0';
  nuovo->info.costo = costo;
  nuovo->info.ore = ore;
  nuovo->info.stato = 'A';
  nuovo->next = NULL;
  p->next = nuovo;                 /* p e` l'ultimo (o il RG)           */
  *pl = rg.next;
  return 1;
}
```

eser3

codice/officina.c

```c
/* ricorsiva: costo totale di tutte le riparazioni (aperte o chiuse) di una targa */
double eser3(TipoListaRip l, char *targa) {
  if (l == NULL)
    return 0;
  if (strcmp(l->info.targa, targa) == 0)
    return l->info.costo + eser3(l->next, targa);
  return eser3(l->next, targa);
}
```

Esecuzione simulata: lista `[AB123CD 120.0 C] -> [EF456GH 300.0 A] -> [AB123CD 250.0 A] -> [IL789MN 800.0 A] -> NULL`, chiamata `t = eser3(l, "AB123CD");`

```
RDA 5: l = NULL                    caso base                 -> 0
RDA 4: l = [IL789MN 800]           targa diversa: eser3(NULL) -> 0
RDA 3: l = [AB123CD 250]           uguale: 250 + eser3(...)  -> 250 + 0   = 250
RDA 2: l = [EF456GH 300]           diversa: eser3(...)       -> 250
RDA 1: l = [AB123CD 120]           uguale: 120 + eser3(...)  -> 120 + 250 = 370
RDA main: t = 370
```

Il programma di prova `officina.c` stampa infatti `totale AB123CD: 370.00`.

Esercizi brevi in più, sulle liste

ESERCIZIO 18.1

Lista di interi. Scrivi `int eliminaTutti(TipoLista *pl, int x)` che elimina **tutte** le occorrenze di x e restituisce quante ne ha eliminate (senza record generatore, gestendo la testa a parte).

Soluzione

```c
int eliminaTutti(TipoLista *pl, int x) {
  TipoNodo *corr, *prec, *d;
  int n = 0;
  while (*pl != NULL && (*pl)->info == x) {   /* 1. elimino in testa finche' serve */
    d = *pl;
    *pl = d->next;
    free(d);
    n++;
  }
  if (*pl == NULL)
    return n;
  prec = *pl;                                 /* 2. ora la testa e` sicuramente != x */
  corr = prec->next;
  while (corr != NULL)
    if (corr->info == x) {
      prec->next = corr->next;
      free(corr);
      corr = prec->next;                      /* prec NON avanza */
      n++;
    } else {
      prec = corr;
      corr = corr->next;
    }
  return n;
}
```

ESERCIZIO 18.2

Scrivi `int inserisciDopo(TipoLista l, int dopo, int x)` che inserisce un nodo con x **subito dopo** il primo nodo che contiene `dopo`. Restituisce 0 se `dopo` non c'è. Perché qui basta `TipoLista` e non `TipoLista *`?

Soluzione

```
int inserisciDopo(TipoLista l, int dopo, int x) {
  TipoNodo *p = l, *nuovo;
  while (p != NULL && p->info != dopo)
    p = p->next;
  if (p == NULL)
    return 0;
  nuovo = malloc(sizeof(TipoNodo));
  if (nuovo == NULL)
    return 0;
  nuovo->info = x;
  nuovo->next = p->next;
  p->next = nuovo;
  return 1;
}
```

Basta `TipoLista` perché la testa **non cambia mai**: il nuovo nodo va sempre dopo un nodo esistente. Si modificano i nodi (che sono nello heap, condivisi), non il puntatore di testa del chiamante.

ESERCIZIO 18.3

Due liste di interi ordinate in modo crescente. Scrivi `TipoLista fondiListe(TipoLista a, TipoLista b)` che crea una **nuova** lista ordinata con tutti gli elementi delle due (le liste originali restano intatte).

Soluzione

Uso il record generatore e un puntatore `coda` all'ultimo nodo della lista risultato, così ogni inserimento in coda costa un solo passo.

```
TipoLista fondiListe(TipoLista a, TipoLista b) {
  TipoNodo rg, *coda = &rg, *n;
  int x;
  rg.next = NULL;
  while (a != NULL || b != NULL) {
    if (b == NULL || (a != NULL && a->info <= b->info)) {
      x = a->info;
      a = a->next;
    } else {
      x = b->info;
      b = b->next;
    }
    n = malloc(sizeof(TipoNodo));
    if (n == NULL)
      break;                    /* memoria finita: restituisco quanto fatto */
    n->info = x;
    n->next = NULL;
    coda->next = n;
    coda = n;
  }
  return rg.next;
}
```

ESERCIZIO 18.4 (RICORSIONE + STRINGHE)

Lista di struct con un campo `char *nome` (stringa esatta). Scrivi una funzione **ricorsiva** `TipoNodoElem *cercaNome(TipoListaElem l, char *nome)` che restituisce il puntatore al nodo con quel nome, o `NULL`.

Soluzione

```
TipoNodoElem *cercaNome(TipoListaElem l, char *nome) {
  if (l == NULL)
    return NULL;                              /* non trovato   */
  if (strcmp(l->info.nome, nome) == 0)
    return l;                                 /* trovato       */
  return cercaNome(l->next, nome);            /* cerca nel resto */
}
```

CAPITOLO 19

Preparazione all'orale

All'orale si **discute lo scritto** (sappi spiegare ogni riga che hai scritto e correggere da solo gli errori) e si fanno **domande sul programma**. Qui sotto le domande più probabili con una risposta breve: allenati a ripeterle **a voce**, con un disegno o un esempio.

Architettura e rappresentazione

Cos'è l'architettura di Von Neumann?

Uno schema di calcolatore in cui programma e dati stanno nella **stessa memoria** centrale; una **CPU** (unità di controllo, ALU, registri) preleva ed esegue le istruzioni una alla volta; memoria e periferiche sono collegate da un **bus**.

Cosa fanno PC e IR? Cos'è il ciclo fetch-decode-execute?

Il **PC** contiene l'indirizzo della prossima istruzione, l'**IR** l'istruzione in esecuzione. **Fetch**: si copia in IR l'istruzione all'indirizzo del PC e si incrementa il PC; **decode**: la CU riconosce il codice operativo; **execute**: si esegue. Un salto scrive un nuovo indirizzo nel PC.

Differenza tra compilatore, linker e interprete?

Il **compilatore** traduce tutto il programma sorgente in codice oggetto (linguaggio macchina), segnalando gli errori; il **linker** unisce il codice oggetto con quello delle librerie e produce l'eseguibile. Un **interprete** invece traduce ed esegue un'istruzione alla volta, senza produrre un eseguibile.

Perché si usa il complemento a due?

Ha **un solo zero**, rappresenta $[-2^{n-1}, 2^{n-1}-1]$, e la **sottrazione diventa una somma** (N - M = N + (-M)), quindi basta un solo circuito sommatore.

Perché i float sono approssimati? Cos'è l'eccesso 127?

I bit sono finiti (23 di mantissa) e molte frazioni in binario sono **periodiche** (0.1, 0.37...): le cifre in più si perdono. L'esponente è memorizzato sommandogli 127, così con 8 bit senza segno (0-255) si rappresentano esponenti anche negativi.

Linguaggio C

Cos'è un record di attivazione?

La zona di memoria (sulla pila) allocata **a ogni chiamata** di funzione: contiene parametri formali, variabili locali, valore e indirizzo di ritorno. Viene deallocata quando la funzione termina. Ogni chiamata ricorsiva ha il suo RDA.

Come si passano i parametri in C?

Sempre **per valore**: la funzione riceve una copia. Per modificare una variabile del chiamante si passa il suo **indirizzo** (parametro puntatore, "parametro di output"). Gli array sembrano passati per riferimento perché il loro nome è già l'indirizzo del primo elemento.

Differenza tra variabile locale, globale, static?

Locale: visibile nel blocco, vive durante l'attivazione. **Globale**: visibile ovunque dopo la dichiarazione, vive per tutto il programma. **Locale static**: visibile nella funzione ma conserva il valore tra le chiamate.

Che relazione c'è tra array e puntatori?

Il nome dell'array è l'indirizzo del primo elemento; `v[i]` equivale a `*(v+i)` (aritmetica dei puntatori: `+1` avanza di un elemento, non di un byte). Ma un array non si può riassegnare né incrementare; un puntatore sì.

Cos'è una stringa in C? Perché `malloc(strlen(s)+1)`?

Un array di char terminato da `'\0'`. `strlen` conta i caratteri **senza** il terminatore, quindi serve un posto in più per il `'\0'`.

Differenza tra allocazione statica e dinamica?

Statica: dimensione decisa nel codice, memoria gestita automaticamente (variabili locali nella pila, globali). **Dinamica**: memoria chiesta durante l'esecuzione con `malloc` nello **heap**, della dimensione che serve; resta finché non si chiama `free`. Rischi: `NULL` non controllato, memory leak, puntatori pendenti.

Cosa sono le stringhe "esatte" e perché si usano?

Stringhe allocate dinamicamente con esattamente `strlen + 1` caratteri: non si spreca memoria come con array sovradimensionati. Si legge in un buffer grande, si alloca la dimensione esatta, si copia.

Differenza tra file di testo e file binario?

Il file di testo contiene caratteri (leggibili, `fprintf` / `fscanf`); il binario contiene i byte come sono in memoria (`fwrite` / `fread`), è più compatto e permette l'accesso diretto con `fseek` a record di dimensione fissa.

Struct per valore o per indirizzo?

Per valore la funzione lavora su una copia (e copiare una struct grande costa); per indirizzo può modificare l'originale e si usa la freccia `p->campo`.

Metodologia e strutture dati

Cos'è la ricorsione? Cosa serve perché termini?

Una funzione che chiama se stessa. Servono un **caso base** risolto direttamente e un **caso ricorsivo** che si riduce a un problema più piccolo, avvicinandosi al caso base.

Ricorsione o iterazione?

Sono equivalenti in potenza. La ricorsione è più naturale per strutture ricorsive (liste, alberi) e per il divide et impera, ma costa un RDA per chiamata (tempo e memoria, rischio di stack overflow).

Come funzionano selection, bubble e insertion sort? Quanto costano?

Selection: cerca il minimo del resto e lo scambia in posizione. Bubble: confronta e scambia vicini, il più grande sale in fondo. Insertion: inserisce ogni elemento al suo posto nella parte ordinata. Tutti circa n^2 confronti nel caso peggiore; merge sort n log n.

Quando si può usare la ricerca binaria?

Solo su dati **ordinati** e con accesso diretto (array). Dimezza lo spazio di ricerca a ogni passo: circa $\log_2 n$ confronti.

Scatola nera e scatola trasparente: differenze?

Nera: test scelti dalla **specifica** (classi di equivalenza dell'input, casi limite), senza guardare il codice. Trasparente: test scelti **guardando il codice** per coprire istruzioni, decisioni, condizioni, valori limite. Nessuno dei due garantisce l'assenza di errori.

Cos'è il test top-down? Cos'è uno stub?

Test incrementale di un programma modulare: si prova una funzione alla volta partendo dal main; le funzioni non ancora provate sono sostituite da **stub** (versioni finte che danno il risultato giusto per il test).

Cos'è un tipo di dato astratto?

Un tipo definito dalle **operazioni** che si possono fare sui dati e dal loro significato, indipendentemente dalla **rappresentazione concreta** (array, lista...). Esempi: tabella, lista.

Lista o array: vantaggi e svantaggi?

Lista: dimensione dinamica, inserimenti ed eliminazioni senza spostare elementi, nessuno spreco; ma niente accesso diretto (per il k-esimo bisogna scorrere) e un puntatore in più per nodo. Array: accesso diretto `v[k]`, ricerca binaria possibile; ma dimensione fissa (o costose `realloc`) e spostamenti per inserire in mezzo.

Perché alcune funzioni sulle liste ricevono `TipoLista *`?

Perché possono **cambiare la testa** (inserimento in testa o ordinato, eliminazione del primo): per modificare il puntatore di testa del chiamante bisogna riceverne l'indirizzo.

Cos'è il record generatore?

Un nodo fittizio messo davanti al primo nodo vero: così ogni nodo ha un predecessore e gli inserimenti/eliminazioni non hanno il caso speciale della testa. Alla fine la testa è `rg.next`.

Cosa sono le visite di un albero?

Algoritmi ricorsivi che esaminano tutti i nodi: preordine (radice, sinistro, destro), inordine (sinistro, radice, destro), postordine (sinistro, destro, radice). In un albero binario di ricerca l'inordine dà i valori ordinati.

Discutere il tuo scritto

Porta all'orale una copia (o la memoria) di cosa hai scritto. Prima dell'orale **riscrivi a casa la tua soluzione corretta** e provala al computer: se all'orale sai dire "qui avevo sbagliato, la versione giusta è questa, e il motivo è questo", fai una buona impressione e recuperi punti.

CAPITOLO 20

Formulario da ripassare il giorno prima

Rappresentazione

Cosa	Formula / regola
binario puro, n bit	$[0, 2^n - 1]$
modulo e segno, n bit	$[-(2^{n-1} - 1), 2^{n-1} - 1]$, due zeri
complemento a 2, n bit	$[-2^{n-1}, 2^{n-1} - 1]$; negativo: inverti i bit e somma 1
float 32 bit	1 segno, 8 esponente (eccesso 127), 23 mantissa: $(-1)^S \times 1.M \times 2^{E-127}$
potenze di 2	2^7=128, 2^8=256, 2^{10}=1024, 2^{11}=2048, 2^{15}=32768, 2^{16}=65536
ASCII	'0'=48, 'A'=65, 'a'=97, '\0'=0

C in una pagina

```
printf("%d %c %f %s %lf\n", ...);   scanf("%d %lf %s", &i, &d, stringa);
/* divisione intera: 7/2 == 3;  reale: (double) a / b                      */
/* stringhe: strlen strcpy strcmp(a,b)==0 strcat  (string.h)              */
/* memoria:  p = malloc(n * sizeof(T)); if (p == NULL) ...; free(p);       */
/* file:     f = fopen(nome, "r"); if (f == NULL) ...; fscanf(...) == 1; fclose(f); */
/* struct:   s.campo   p->campo  == (*p).campo                            */
/* output:   void f(int *out) { *out = ...; (*out)++; }    f(&x);         */
```

Lo schema dei tipi per l'esame

```
typedef struct {
  char *nome;               /* lunghezza ignota -> stringa esatta        */
  char codice[K + 1];       /* K caratteri                               */
  int numero;
  char tipo;
} TipoElemento;

struct nodo { TipoElemento info; struct nodo *next; };
typedef struct nodo TipoNodo;
typedef TipoNodo *TipoLista;
```

Gli schemi sulle liste

```
/* scansione */
for (p = l; p != NULL; p = p->next) ...

/* inserimento in testa */
n = malloc(sizeof(TipoNodo)); n->info = x; n->next = *pl; *pl = n;

/* due puntatori (per inserire prima di / eliminare corr) */
prec = NULL; corr = *pl;
while (corr != NULL && <non e` il posto/nodo giusto>) { prec = corr; corr = corr->next; }

/* agganciare un nuovo nodo prima di corr */
n->next = corr;  if (prec == NULL) *pl = n; else prec->next = n;

/* scavalcare ed eliminare corr */
if (prec == NULL) *pl = corr->next; else prec->next = corr->next;
free(corr->info.nome); free(corr);

/* record generatore */
TipoNodo rg; rg.next = *pl; p = &rg; ... ; *pl = rg.next;

/* ricorsione */
if (l == NULL) return <caso base>;  return <uso l->info> f(l->next);

/* deallocazione */
while (*pl != NULL) { d = *pl; *pl = d->next; free(d); }
```

Test

- **Scatola nera**: dalla specifica; classi di equivalenza dell'input; un rappresentante per classe; casi limite (vuota, un elemento, primo, ultimo, assente, uguale alla soglia).
- **Scatola trasparente**: dal codice; copertura di istruzioni → decisioni (V e F) → condizioni (V e F) → valori limite; cicli con 0, 1, più iterazioni.
- **Test** = dati di prova + output atteso, confronto con l'output effettivo; se diversi, debugging.

Ricorsione

Caso base + caso ricorsivo che si avvicina al caso base. RDA: una per chiamata, impilate; si risolve all'andata fino al caso base, i valori tornano indietro al ritorno.

Il giorno dell'esame

- Documento d'identità, iscrizione su Infostud fatta, homework accettati.
- Leggi tutto il testo, poi i tipi, poi `eser1`, `eser2`, esercizio 3, rilettura con la checklist del capitolo 16.
- Scrivi in modo ordinato e commenta l'algoritmo.

Fonti

Materiale consultato sul sito del corso (prof. Marco Temperini), A.A. 2026/27:

- Pagina del corso e pagina didattica del docente
- Programma 2026/27
- Calendario e contenuto delle lezioni
- Regole e date d'esame e testi d'esame d'esempio
- Homework
- Slide delle lezioni (cartella `tdp-lezioni`), complementi didattici, esercitazioni guidate EG1-EG9
- Scheda del docente su corsidilaurea.uniroma1.it

Al momento della consultazione (novembre 2026) le slide sulle liste (lezioni 21-25) e le EG10-EG11 dell'anno in corso non erano ancora pubblicate: il capitolo sulle liste segue gli argomenti elencati nel calendario delle lezioni e il testo2 (Calvanese et al.), che il corso adotta per quella parte. Quando escono le slide, confrontale con il capitolo 14 e adotta i nomi usati dal professore.

Questo fascicolo è un aiuto allo studio: non sostituisce le lezioni, i libri di testo e le comunicazioni ufficiali su Classroom.